TRƯỜNG ĐẠI HỌC XÂY DỰNG MIỀN TRUNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO KẾT QUẢ THỰC HÀNH 04 LAB
CÔNG NGHỆ PHẦN MỀM

Đề tài : Đề tài 8: Phần mềm Quản lý Thư viện Trường Đại học (ULS)

Mã nhóm / Lớp:	Nhóm 3 - Lớp D23CTC2
Giảng viên hướng dẫn:	TS. Lê Tỷ Khánh
Sinh viên thực hiện:	1. Trần Đình Khôi - MSSV: 23Q74802012006 (Nhóm trưởng) 2. Nguyễn Như Quỳnh - MSSV: 21DQ4802011003
Đường dẫn Repository:	GitHub: https://github.com/khoidev2001/university-library-system
Địa chỉ Deploy Live:	https://muce-library.vercel.app
Ngày hoàn thành:	Ngày 19 tháng 10 năm 2026

PHẦN I: TỔNG QUAN DỰ ÁN VÀ PHÂN CÔNG TƯƠNG TÁC SCRUM

1.1. Mô tả tóm tắt dự án

Thư viện trường đại học còn quản lý sách, bạn đọc và mượn trả bằng sổ sách hoặc bảng tính nên tra cứu chậm, tính phạt dễ sai và không có số liệu thống kê. Đề tài xây dựng Hệ thống Quản lý Thư viện Trường Đại học (ULS) dạng ứng dụng web: quản lý đầu sách và bản sao có mã vạch; cho mượn tại quầy, trả, gia hạn, báo mất và tự tính phạt theo chính sách cấu hình được; đặt trước sách, thông báo và email nhắc hạn; nộp phạt trực tuyến qua VNPay; thống kê, xuất báo cáo, in thẻ và nhãn mã vạch. Hai mô hình AI gợi ý sách cá nhân hoá và dự báo nguy cơ trả trễ. Công nghệ: Next.js 15 với TailwindCSS, NestJS với Prisma và PostgreSQL, FastAPI với scikit-learn, Docker Compose, GitHub Actions, triển khai trên Vercel, Render và Neon.

1.2. Bảng phân công nhiệm vụ và mức độ hoàn thành

Họ và tên SV	Vai trò trong nhóm	Nhiệm vụ phụ trách (LAB 1 - 4)	% Hoàn thành
Trần Đình Khôi	Nhóm trưởng, Scrum Master, Backend và DevOps	SRS và Product Backlog (LAB 1), ERD và đặc tả API (LAB 2), Backend NestJS và Prisma (LAB 3), Docker và CI/CD (LAB 4)	100%
Nguyễn Như Quỳnh	Frontend và AI/ML	Actor và yêu cầu phi chức năng (LAB 1), UML (LAB 2), giao diện Next.js và dịch vụ AI (LAB 3), kiểm thử (LAB 4)	100%

Bảng 1. Phân công nhiệm vụ và mức độ hoàn thành

PHẦN II: KẾT QUẢ THỰC HÀNH TỪNG BÀI LAB

2.1. LAB 01: Phân tích yêu cầu & Thiết kế Đặc tả Phần mềm (SRS)

a) Xác định Actor: Danh sách Actor trong hệ thống:

- **Reader (Bạn đọc: sinh viên, giảng viên):** đăng ký, đăng nhập; tìm kiếm và xem chi tiết sách; xem sách đang mượn, lịch sử, tiền phạt; tự gia hạn; đặt trước sách hết bản; nhận thông báo và email nhắc hạn; nộp phạt trực tuyến qua VNPay; đánh giá sách; nhận gợi ý AI; đổi mật khẩu, sửa hồ sơ. Chỉ truy cập dữ liệu của chính mình.
- **Librarian (Thủ thư):** quyền tra cứu như Reader; CRUD đầu sách, tác giả, thể loại, bản sao; cho mượn tại quầy bằng mã vạch, nhận trả, báo mất, thu phạt tiền mặt; quản lý hàng đợi đặt trước; tạo bạn đọc tại quầy, khoá và mở thẻ, cấp lại mật khẩu tạm; xem báo cáo, xuất CSV hoặc Excel, in thẻ và nhãn mã vạch, nhập sách từ tệp; thấy dự báo nguy cơ trả trễ khi cho mượn.
- **Admin (Quản trị):** toàn bộ quyền Librarian; tạo tài khoản Librarian và Admin; cấu hình chính sách mượn (số cuốn, hạn, gia hạn, mức phạt) theo loại bạn đọc; nhập sách hàng loạt theo ISBN; huấn luyện lại mô hình AI.
- **Actor phụ (hệ thống ngoài):** AI Service (gợi ý và dự báo), Google Books API (thông tin sách theo ISBN), Resend (gửi email), VNPay (cổng thanh toán, môi trường sandbox).

b) Product Backlog & User Stories: Trích xuất danh sách Product Backlog chính (37 User Story, 155 điểm; US01 đến US20 là MVP, US21 đến US37 là các đợt mở rộng; Acceptance Criteria rút gọn, đầy đủ trong SRS):

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US01	As a Reader, I want to đăng ký tài khoản bằng email, mật khẩu, mã SV/GV và loại bạn đọc, so that tôi có thể dùng thư viện trực tuyến.	1. Email và mã SV/GV chưa tồn tại → tạo tài khoản READER, trả 201. 2. Email trùng → 409 `EMAIL_EXISTS`. 3. Mật khẩu < 8 ký tự → 400.	3 pt
US02	As a người dùng, I want to đăng nhập bằng email và mật khẩu, so that hệ thống nhận diện vai trò của tôi.	1. Đúng thông tin → trả JWT hạn 24h + thông tin user. 2. Sai mật khẩu → 401, không tiết lộ email có tồn tại hay không. 3. Tài khoản bị khoá → 403 `ACCOUNT_LOCKED`.	2 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US03	As a Librarian, I want to thêm, sửa, xoá đầu sách với tác giả và thể loại, so that kho sách luôn đúng thực tế.	1. Thêm sách thiếu tiêu đề → 400. 2. ISBN trùng → 409. 3. Sách gắn được ≥ 1 tác giả và 1 thể loại.	5 pt
US04	As a Librarian, I want to quản lý bản sao vật lý (mã vạch, trạng thái, vị trí kệ), so that biết chính xác cuốn nào đang ở đâu.	1. Mã vạch duy nhất, trùng → 409. 2. Bản sao mới có trạng thái `AVAILABLE`. 3. Không đổi sang `AVAILABLE` khi bản sao có phiếu `ACTIVE` → 409.	3 pt
US05	As a Reader, I want to tìm sách theo tiêu đề, tác giả, thể loại hoặc ISBN và lọc sách còn mượn được, so that tôi tìm nhanh cuốn cần.	1. Từ khoá khớp một phần tiêu đề/tác giả/ISBN, không phân biệt hoa thường. 2. Lọc theo thể loại, tác giả, `available=true`. 3. Phân trang 20/trang, trả tổng số.	5 pt
US06	As a Reader, I want to xem chi tiết một đầu sách và số bản còn, so that tôi biết có thể mượn ngay không.	1. Hiển thị đủ thông tin, tác giả, thể loại, ảnh bìa. 2. Hiển thị "Còn X/Y bản". 3. Hiển thị điểm đánh giá trung bình.	2 pt
US07	As a Librarian, I want to tạo phiếu mượn bằng mã vạch bản sao và mã bạn đọc, so that cho mượn tại quầy nhanh và đúng quy định.	1. Đủ điều kiện → phiếu `ACTIVE`, `due_at` = hôm nay + `loan_days` theo loại bạn đọc, bản sao → `BORROWED`. 2. Đã đủ `max_books` → 409 `LOAN_LIMIT_REACHED`. 3. Đang có sách quá hạn → 409 `HAS_OVERDUE_LOAN`.	8 pt
US08	As a Librarian, I want to nhận trả sách theo mã vạch và hệ thống tự tính phạt, so that không phải tính tay và không bỏ sót.	1. Phiếu → `RETURNED`, bản sao → `AVAILABLE`. 2. Trả đúng hạn → không có phạt. 3. Trả trễ N ngày → tạo phạt `OVERDUE` = N × `fine_per_day`, hiển thị số tiền ngay.	5 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US09	As a Reader, I want to tự gia hạn sách đang mượn, so that không phải đến quầy.	1. Đủ điều kiện → `due_at` += `renew_extra_days`, `renewed_count` += 1. 2. Đã gia hạn đủ số lần → 409 `RENEW_LIMIT_REACHED`. 3. Đã quá hạn → 409 `LOAN_OVERDUE_CANNOT_RENEW`.	3 pt
US10	As a Reader, I want to xem sách đang mượn, ngày hạn và lịch sử mượn, so that tôi trả đúng hạn.	1. Danh sách phiếu `ACTIVE` với số ngày còn lại; quá hạn tô đỏ kèm số ngày trễ. 2. Lịch sử phiếu `RETURNED`/`LOST` phân trang. 3. Chỉ thấy phiếu của mình.	3 pt
US11	As a Librarian, I want to ghi nhận sách bị mất, so that kho và công nợ bạn đọc được cập nhật đúng.	1. Phiếu → `LOST`, bản sao → `LOST`. 2. Tạo phạt `LOST` = giá bìa + phạt trễ đến ngày báo. 3. Bản sao `LOST` không xuất hiện trong số "còn mượn được".	3 pt
US12	As a Reader, I want to xem tiền phạt của mình; as a Librarian, I want to ghi nhận đã thu tiền, so that công nợ minh bạch.	1. Reader thấy từng khoản (loại, số tiền, phiếu liên quan) và tổng chưa trả. 2. Librarian lọc khoản chưa trả, bấm "Đã thu" → `paid_at`, người thu. 3. Khoản đã trả không thu lại được → 409.	3 pt
US13	As a Librarian, I want to tra cứu bạn đọc, xem hồ sơ và khoá/mở thẻ, so that xử lý được vi phạm.	1. Tìm theo tên/email/mã, lọc theo trạng thái thẻ. 2. Hồ sơ: sách đang mượn, lịch sử, phạt chưa trả. 3. Khoá thẻ → không mượn, không đăng nhập; mở lại được.	5 pt
US14	As an Admin, I want to tạo tài khoản thủ thư, so that nhân viên mới có thể làm việc.	1. Tạo user với vai trò LIBRARIAN hoặc ADMIN. 2. Librarian gọi endpoint này → 403. 3. Email trùng → 409.	2 pt
US15	As an Admin, I want to cấu hình số cuốn tối đa, hạn mượn, số lần gia hạn và mức phạt theo loại bạn đọc, so that thư viện đổi quy định không cần sửa code.	1. Xem bảng chính sách STUDENT/LECTURER. 2. Sửa với giá trị nguyên dương; giá trị ≤ 0 → 400. 3. Phiếu tạo sau đó dùng chính sách mới; phiếu cũ giữ nguyên hạn.	3 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US16	As a Librarian, I want to xem dashboard thống kê (sách mượn nhiều, bạn đọc quá hạn, lượt mượn theo tháng, tồn kho theo thể loại), so that đề xuất mua sách và nhắc bạn đọc.	1. Top 10 sách mượn nhiều theo khoảng thời gian chọn. 2. Danh sách quá hạn kèm số ngày trễ, phạt ước tính. 3. Biểu đồ cột lượt mượn 12 tháng.	5 pt
US17	As a Reader, I want to chấm điểm 1 đến 5 sao cho sách đã mượn, so that gợi ý cho tôi chính xác hơn.	1. Chỉ chấm được sách có phiếu `RETURNED` của mình; chưa mượn → 403 `NOT_BORROWED`. 2. Điểm ngoài 1 đến 5 → 400. 3. Chấm lại → ghi đè, không tạo bản ghi mới.	2 pt
US18	As a Reader, I want to nhận danh sách sách gợi ý theo sở thích, so that tôi khám phá sách phù hợp.	1. Trả 10 sách, không gồm sách đã mượn. 2. Bạn đọc có lịch sử → kết quả từ mô hình lai; không có → sách mượn nhiều nhất. 3. Precision@5 trên tập kiểm tra ≥ 0,15 (báo cáo kèm số đo).	8 pt
US19	As a Librarian, I want to thấy nguy cơ trả trễ của bạn đọc ngay khi tạo phiếu mượn, so that tôi nhắc nhở người có nguy cơ cao.	1. Phản hồi tạo phiếu kèm `overdueRisk` 0 đến 1; UI hiện % và màu (xanh < 30, vàng 30 đến 60, đỏ > 60). 2. AI không phản hồi trong 2 s → phiếu vẫn tạo, `overdueRisk = null`, UI hiện "không có dự báo". 3. Mô hình đạt ROC-AUC ≥ 0,70 trên tập kiểm tra (báo cáo kèm bảng so sánh Logistic Regression vs Random Forest).	5 pt
US20	As a Librarian, I want to quản lý danh mục tác giả và thể loại, so that nhập sách nhất quán.	1. CRUD tác giả, thể loại. 2. Xoá thể loại còn sách → 409 `CATEGORY_IN_USE`. 3. Tên thể loại duy nhất.	2 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US21	As a Reader, I want to đặt trước một cuốn sách đang hết bản, so that tôi được giữ sách khi có người trả.	1. Sách không còn bản AVAILABLE nhưng còn bản đang lưu hành → 201, đặt trước WAITING. 2. Sách còn bản trên kệ → 409 BOOK_AVAILABLE; sách không có bản nào → 409 NO_COPIES. 3. Đã đặt cuốn đó → 409 ALREADY_RESERVED; đang mượn cuốn đó → 409 ALREADY_BORROWING.	5 pt
US22	As a Reader, I want to xem vị trí trong hàng đợi và huỷ đặt trước, so that tôi biết còn phải chờ bao lâu và nhả lượt khi không cần.	1. Danh sách đặt trước của tôi hiện queuePosition (WAITING từ 1, READY là 0) và hạn nhận sách của đặt trước READY. 2. Huỷ đặt trước của mình → CANCELLED. 3. Huỷ đặt trước đã kết thúc → 409 RESERVATION_NOT_ACTIVE.	2 pt
US23	As a Librarian, I want to xem hàng đợi đặt trước và huỷ khi cần, so that xử lý được trường hợp bạn đọc không đến nhận.	1. Danh sách có phân trang, lọc theo đầu sách và trạng thái, mặc định WAITING và READY. 2. Huỷ được đặt trước của bất kỳ bạn đọc nào. 3. Reader gọi danh sách này → 403.	2 pt
US24	As a Reader, I want to được giữ sách trong 3 ngày khi có người trả, so that tôi đến quầy là mượn được.	1. Khi bản sao được trả, người đứng đầu hàng đợi có thẻ hoạt động nhận bản sao HELD, đặt trước READY, expires_at = +3 ngày, thông báo RESERVATION_READY. 2. Người khác mượn bản HELD → 409 COPY_HELD; đúng người đặt mượn được và đặt trước thành FULFILLED. 3. Quá hạn nhận → EXPIRED, thông báo RESERVATION_EXPIRED, bản sao chuyển cho người kế tiếp hoặc AVAILABLE.	5 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US25	As a Reader, I want to nhận thông báo trong web về sách sắp đến hạn, quá hạn và đặt trước, so that tôi không quên trả sách.	1. Chuông hiện số chưa đọc, làm mới mỗi 60 giây. 2. Phiếu có hạn trong 2 ngày tới sinh một thông báo `LOAN_DUE_SOON`; phiếu sang ngày sau hạn sinh một `LOAN_OVERDUE`; mỗi phiếu và loại chỉ một lần dù việc định kỳ chạy lại. 3. Đánh dấu đã đọc một hoặc tất cả; không đánh dấu được thông báo của người khác → 404.	3 pt
US26	As a Reader, I want to nhận email nhắc hạn và tắt được khi không muốn, so that tôi được nhắc cả khi không mở web.	1. Bạn đọc bật nhắc hạn nhận một email gộp các phiếu cùng loại. 2. Mỗi phiếu và loại chỉ gửi thành công một lần; lỗi gửi thử lại tối đa 5 lần. 3. Tắt tại `/account` thì không nhận thêm.	5 pt
US27	As a người dùng, I want to đổi mật khẩu và sửa hồ sơ, so that tôi tự quản lý tài khoản.	1. Đổi mật khẩu cần mật khẩu hiện tại đúng, sai → 400 `WRONG_CURRENT_PASSWORD`. 2. Mật khẩu mới trùng mật khẩu cũ → 400 `PASSWORD_UNCHANGED`; ngắn hơn 8 ký tự hoặc quá 72 byte → 400. 3. Sửa họ tên và cờ email nhắc hạn; gửi email, vai trò, mã thẻ → 400.	3 pt
US28	As a người dùng, I want to đặt lại mật khẩu qua email khi quên, so that tôi vào lại được mà không cần đến quầy.	1. Quên mật khẩu luôn trả 200 cùng nội dung dù email có tồn tại hay không. 2. Email thật nhận liên kết có token hạn 1 giờ, dùng một lần; hai yêu cầu cách nhau dưới 60 giây thì không gửi thư thứ hai. 3. Token sai, hết hạn hoặc đã dùng → 400 `INVALID_RESET_TOKEN`.	5 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US29	As a Librarian, I want to tạo tài khoản bạn đọc tại quầy và cấp lại mật khẩu tạm, so that bạn đọc không có email vẫn dùng được hệ thống.	1. Tạo bạn đọc với email, họ tên, loại bạn đọc; mã thẻ bỏ trống thì tự sinh `DGyy` + 6 số. 2. Phản hồi có mật khẩu tạm 12 ký tự, chỉ một lần. 3. Bạn đọc đăng nhập bằng mật khẩu tạm chỉ gọi được `/auth/me` và đổi mật khẩu, các API khác → 403 `PASSWORD_CHANGE_REQUIRED`.	5 pt
US30	As a Reader, I want to thanh toán tiền phạt trực tuyến qua VNPay, so that tôi không phải đến quầy nộp tiền mặt.	1. Chọn khoản phạt (hoặc tất cả) → nhận `paymentUrl` ký HMAC-SHA512; số tiền do máy chủ tính. 2. Dưới 5.000 đồng → 409 `PAYMENT_AMOUNT_TOO_SMALL`; không có khoản phạt → 409 `NO_UNPAID_FINES`. 3. IPN thành công đặt `paid_at` và giao dịch PAID; IPN gửi lại không thu hai lần (`RspCode` 02); sai chữ ký → 97; sai số tiền → 04.	8 pt
US31	As a Librarian, I want to huỷ giao dịch chờ và nhận biết giao dịch cần hoàn tiền, so that không thu trùng và xử lý được tiền về muộn.	1. Thu tiền mặt khoản phạt đang nằm trong giao dịch chờ còn hạn → 409 `FINE_PAYMENT_PENDING`. 2. Huỷ giao dịch chờ → CANCELLED, sau đó thu tiền mặt được. 3. IPN thành công đến sau khi khoản phạt đã được thu tiền mặt → không thu lần hai, giao dịch `needsRefund = true`, IPN vẫn trả 00.	3 pt
US32	As a Librarian, I want to xuất báo cáo ra CSV hoặc Excel, so that tôi lưu, lọc và gửi số liệu cho lãnh đạo.	1. Xuất được 7 bộ dữ liệu (quá hạn, phiếu mượn, sách mượn nhiều, tồn kho, khoản phạt, bạn đọc, danh mục sách) ở cả hai định dạng. 2. CSV mở trong Excel hiển thị đúng tiếng Việt (UTF-8 có BOM); ô bắt đầu bằng `=`, `+`, `-`, `@` được thêm dấu `'`. 3. Quá 100.000 dòng → tệp bị cắt, header `X-Export-Truncated: true`.	5 pt

ID	User Story	Acceptance Criteria (Điều kiện nghiệm thu)	Points
US33	As a Librarian, I want to in thẻ bạn đọc có mã vạch và mã QR, so that bạn đọc dùng thẻ vật lý tại quầy.	1. In một thẻ hoặc hàng loạt tối đa 300 thẻ. 2. Thẻ 85,6 x 54 mm, 10 thẻ trên một tờ A4. 3. Mã Code 128 và QR đều chứa mã thẻ và giải mã ngược được.	3 pt
US34	As a Librarian, I want to in nhãn mã vạch cho bản sao, so that dán lên sách và quét khi cho mượn, trả.	1. Chọn theo đầu sách hoặc bản sao, tối đa 2.000 nhãn, vượt → 422 `TOO_MANY_LABELS`. 2. Có các bố cục 3x8, 2x7, 4x10 và cuộn 50 x 25 mm. 3. Bỏ trống N ô đầu để dùng tiếp tờ in dở.	3 pt
US35	As a Librarian, I want to nhập hàng loạt sách và bản sao từ tệp CSV hoặc xlsx có xem trước, so that nhập kho nhanh và ít sai.	1. Tải tệp mẫu. 2. `dryRun=true` trả xem trước từng dòng và không ghi gì. 3. Ghi thật trả `created`, `updated`, `skipped`, `failed`; tải được các dòng lỗi.	8 pt
US36	As a Librarian, I want to thao tác mượn, trả, thu phạt đúng ngay cả khi nhiều quầy cùng làm, so that dữ liệu không bị sai lệch.	1. Năm bạn đọc mượn cùng một bản sao đồng thời: đúng một phiếu. 2. Một bạn đọc mượn nhiều bản đồng thời: không vượt `max_books`. 3. Trả cùng một phiếu 4 lần đồng thời: một lần 200, một khoản phạt.	5 pt
US37	As a Reader, I want to dùng giao diện rõ ràng, nhanh và đọc được trên điện thoại, so that tôi tra cứu và thao tác thuận tiện.	1. Không cuộn ngang ở 390, 768, 1440 px. 2. Mọi cặp chữ và nền đạt tương phản WCAG AA. 3. Hộp thoại giữ focus, Esc đóng, trả focus; có viền focus cho bàn phím.	8 pt

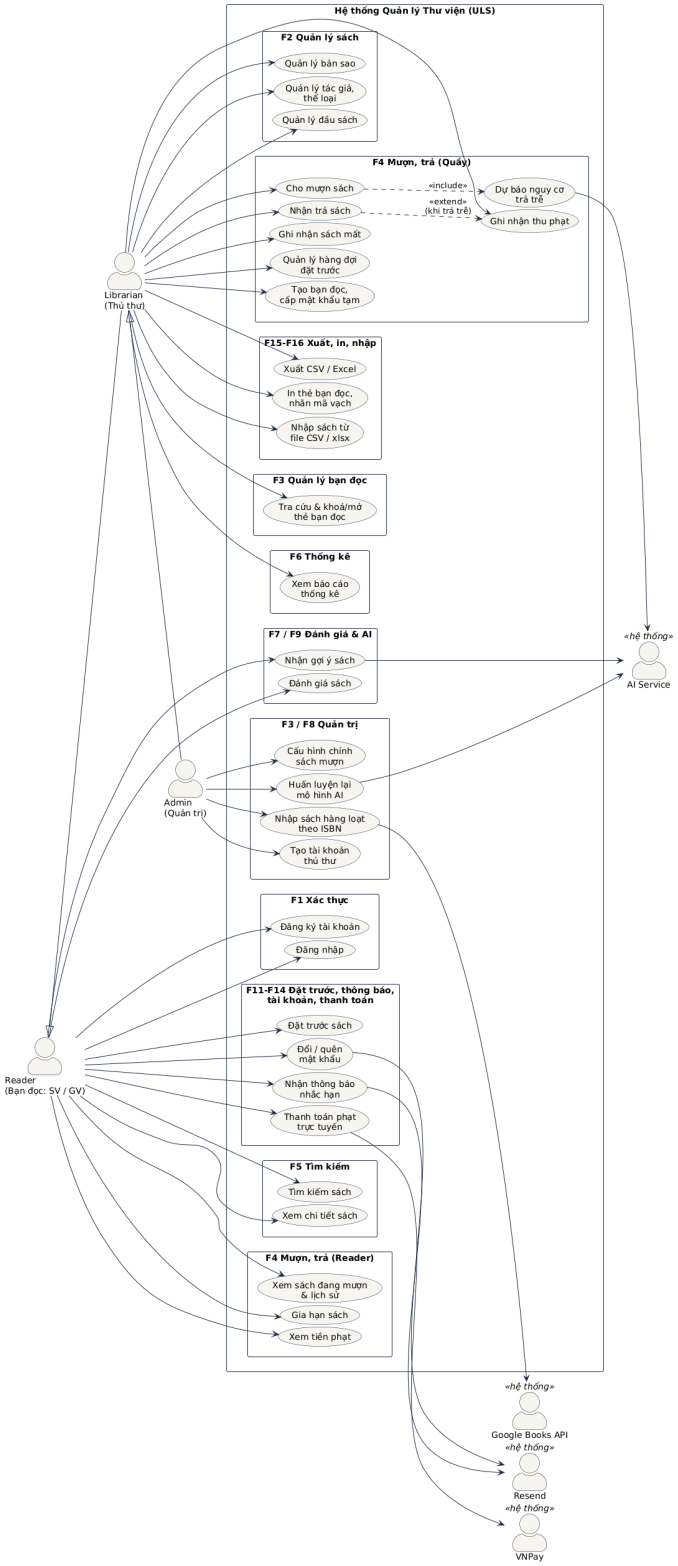
Bảng 2. Product Backlog và User Stories

c) Tài liệu SRS: Link đường dẫn tới file tài liệu đặc tả SRS đầy đủ chuẩn IEEE 830 (phiên bản 2.0): https://github.com/khoidev2001/university-library-system/blob/main/`docs/srs/SRS.md`

2.2. LAB 02: Thiết kế Kiến trúc Hệ thống & Mô hình hoá UML

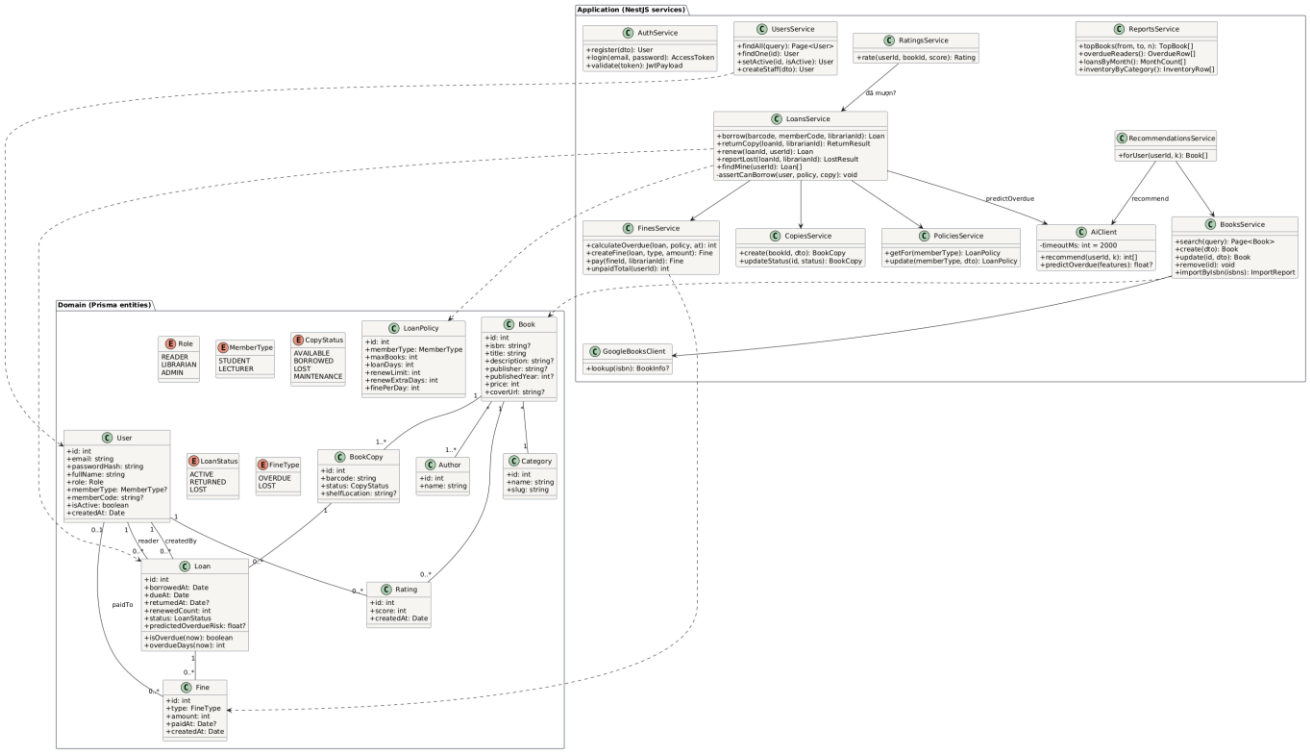
a) Biểu đồ Use Case Tổng Quan: ba actor chính (Reader, Librarian, Admin) với quyền lồng nhau Admin gồm quyền Librarian, Librarian gồm quyền Reader; bốn actor phụ là AI Service, Google Books, Resend và VNPay. Luồng chính: thủ thư quét mã vạch bản

sao và mã bạn đọc, hệ thống kiểm tra chính sách mượn (BR-01 đến BR-05), gọi AI dự báo trả trễ (include) rồi tạo phiếu; khi nhận trả trễ, hệ thống tự tạo khoản phạt (extend).



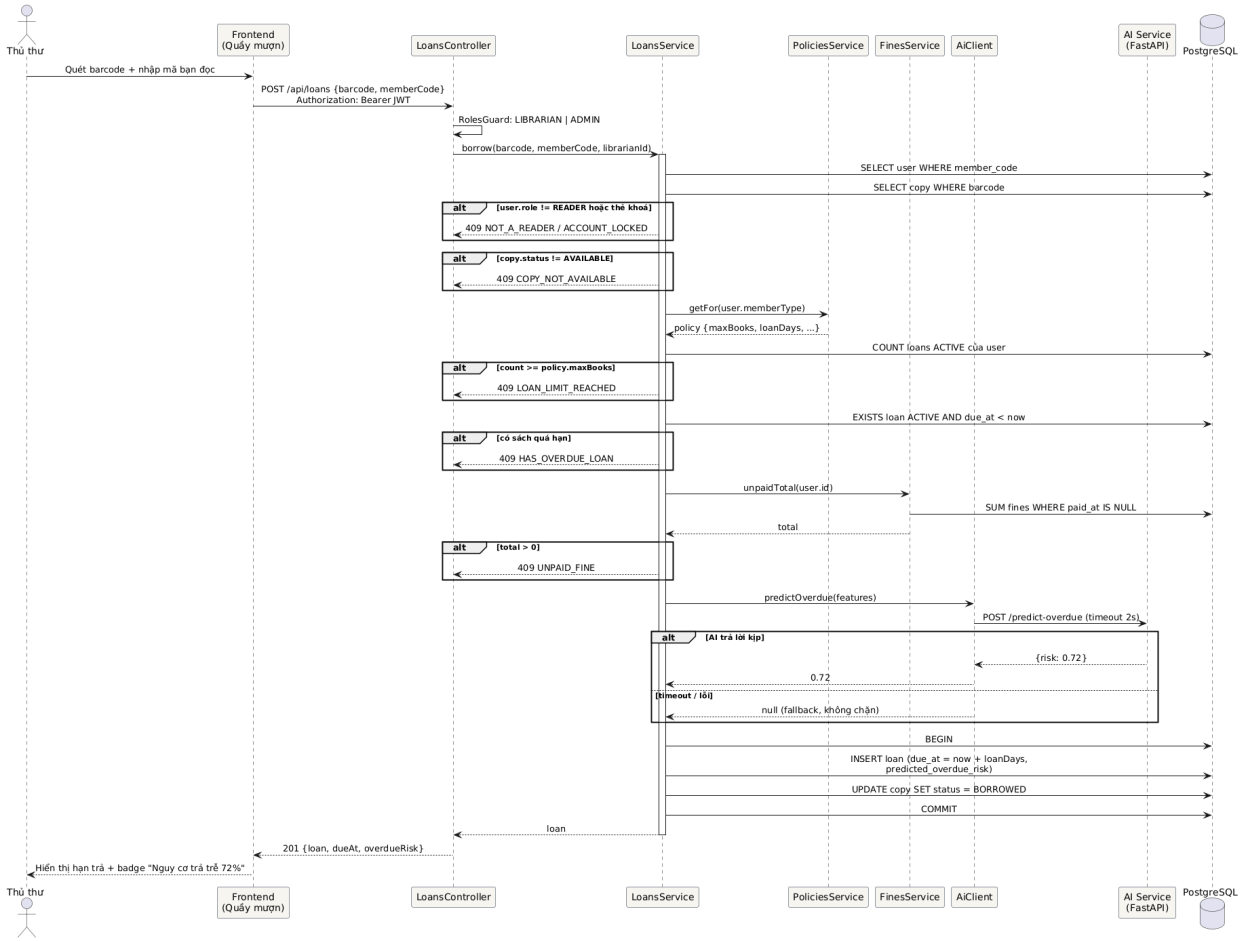
Hình 1. Use Case tổng quan của hệ thống ULS

b) Biểu đồ Lớp (Class Diagram): gói Domain gồm các entity ánh xạ một-một với Prisma schema (User, LoanPolicy, Book, Author, Category, BookCopy, Loan, Fine, Rating, Reservation, Notification, Payment và các enum); gói Application gồm các service NestJS và các client ngoài (AiClient, GoogleBooksClient, MailSender, VNPay). `LoansService` là trung tâm nghiệp vụ, phụ thuộc `PoliciesService`, `FinesService`, `CopiesService`, `ReservationsService` và `AiClient`.

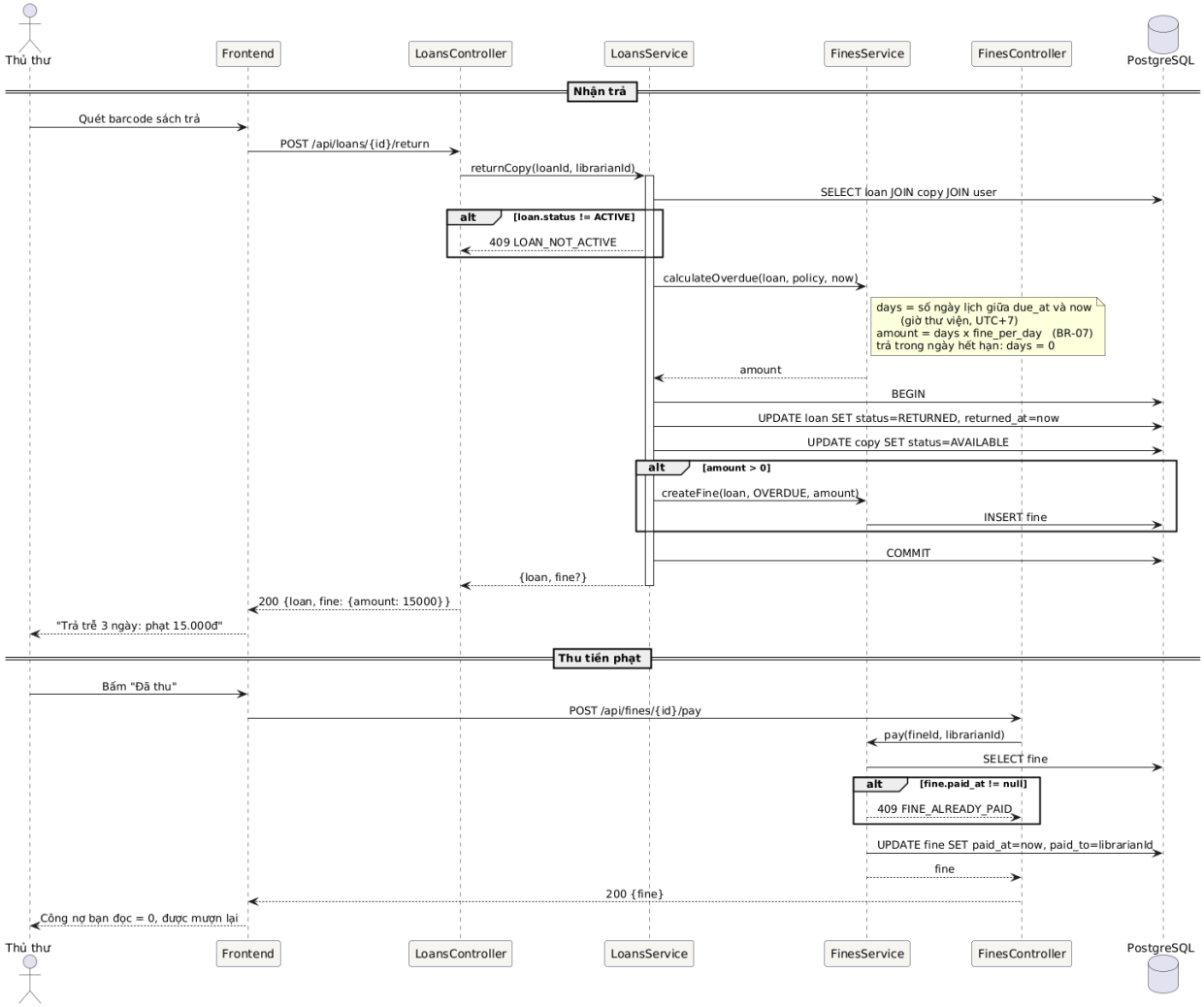


Hình 2. Biểu đồ lớp của hệ thống

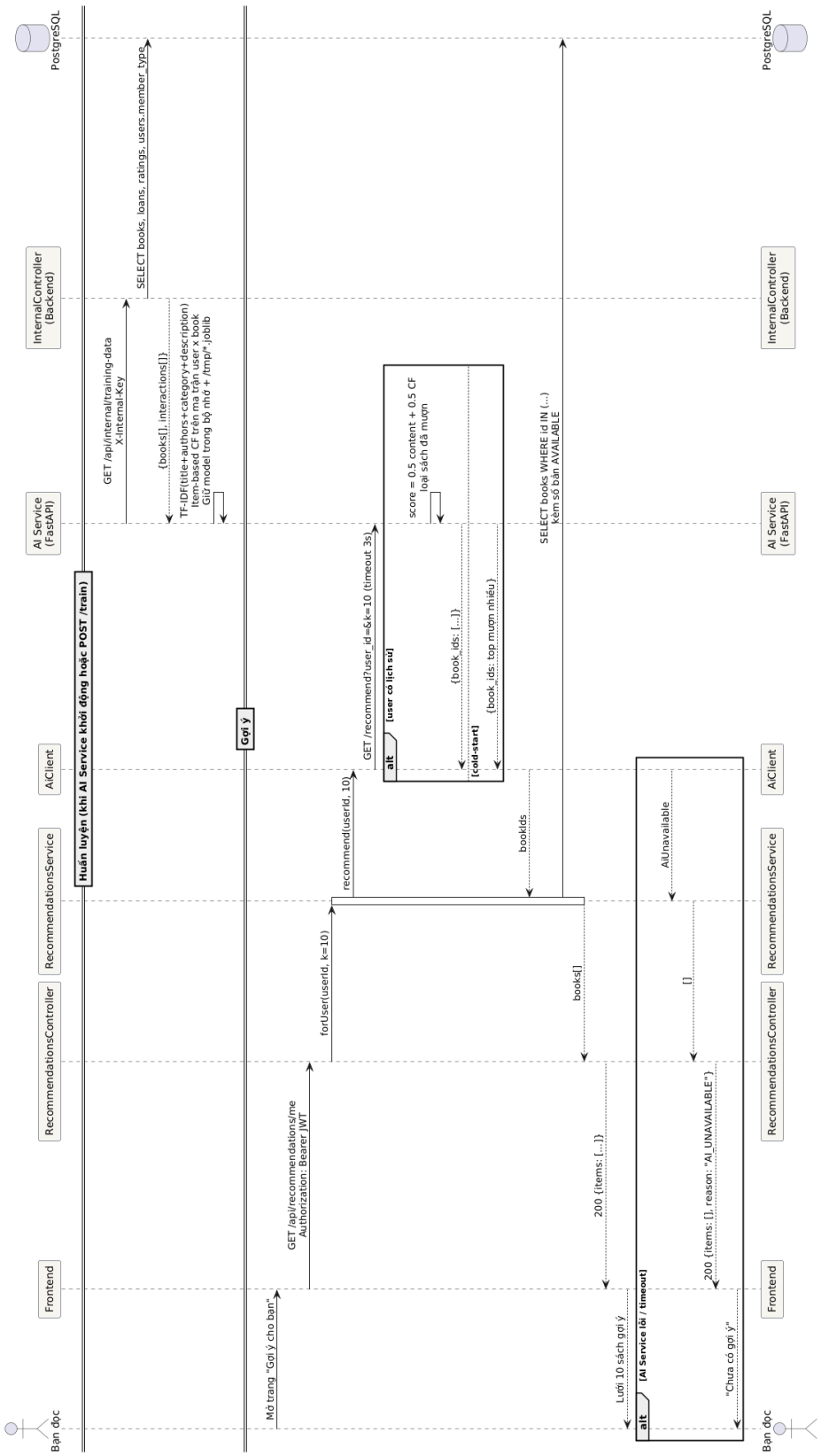
c) Biểu đồ Tuần tự (Sequence Diagram): luồng quan trọng nhất là cho mượn sách tại quầy kèm dự báo AI (US07, US19): 5 điều kiện chặn BR-01 đến BR-05, gọi AI Service với thời hạn chờ 2 giây (dự phòng risk = `null`), một giao dịch tạo phiếu và đổi trạng thái bản sao. Các luồng còn lại: nhận trả, tính phạt và thu tiền; gợi ý sách; đặt trước sách; nộp phạt trực tuyến qua VNPay.



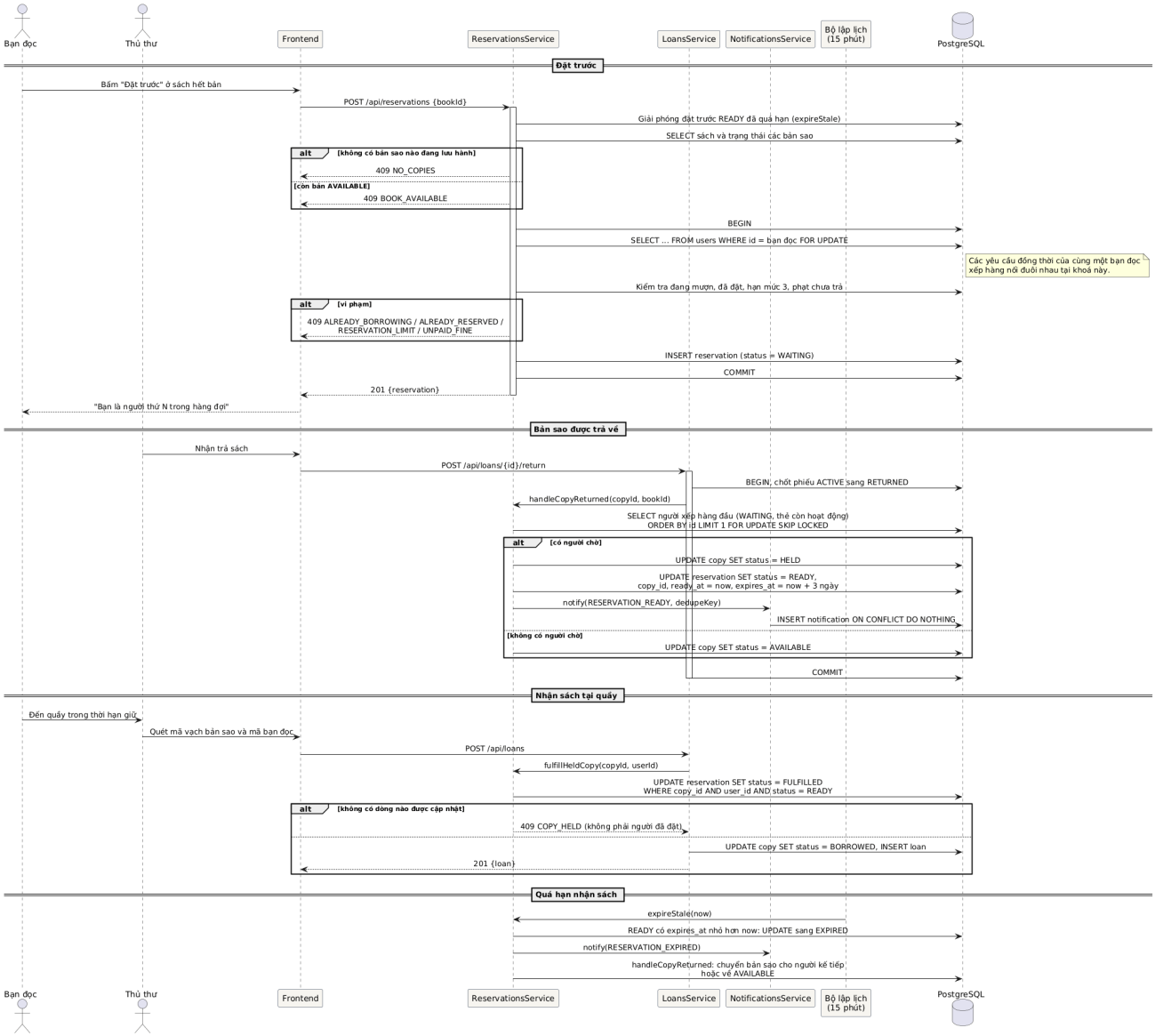
Hình 3. Sequence: cho mượn sách tại quầy kèm dự báo AI (US07, US19)



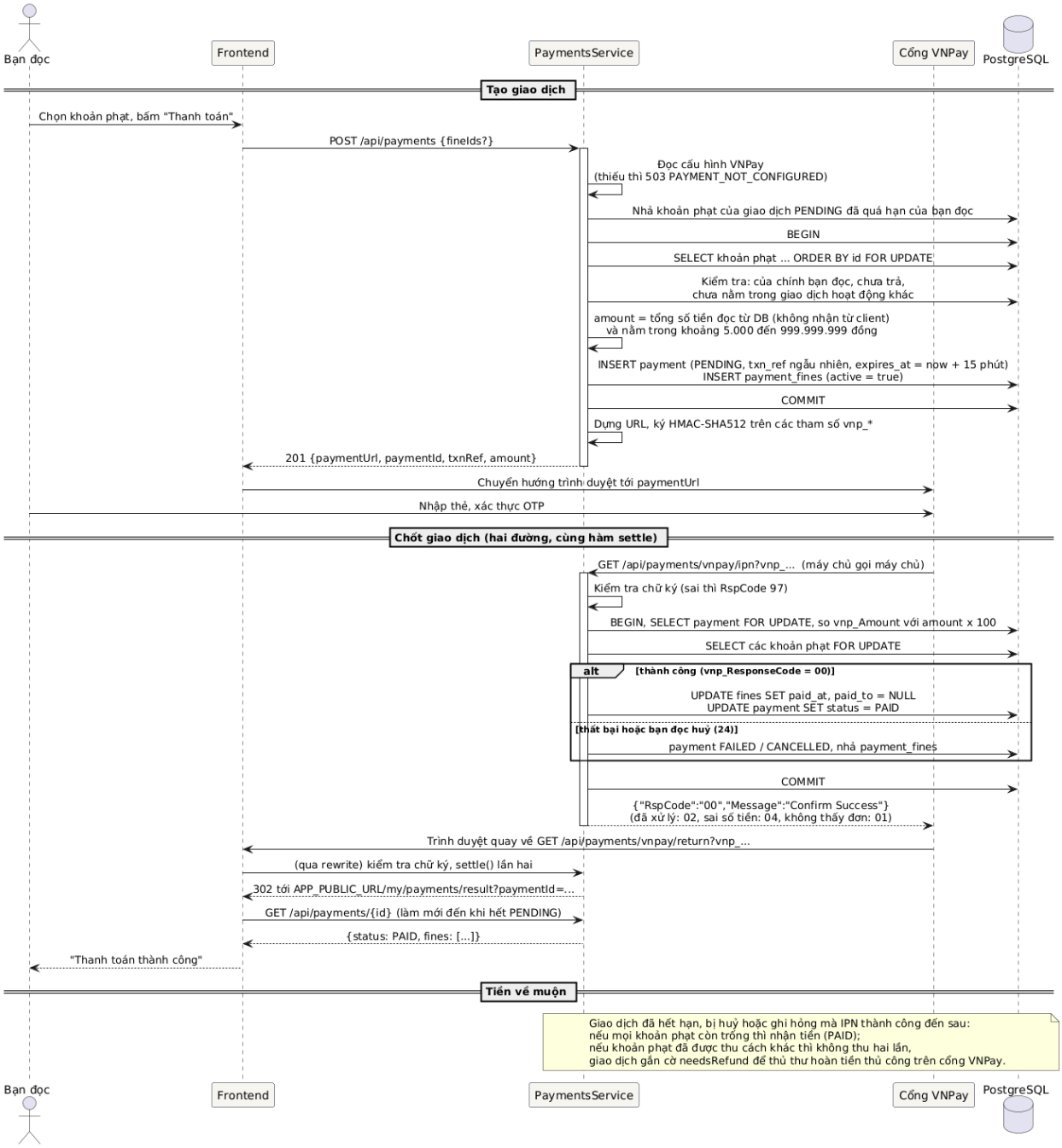
Hình 4. Sequence: nhận trả sách, tính phạt, thu tiền (US08, US12)



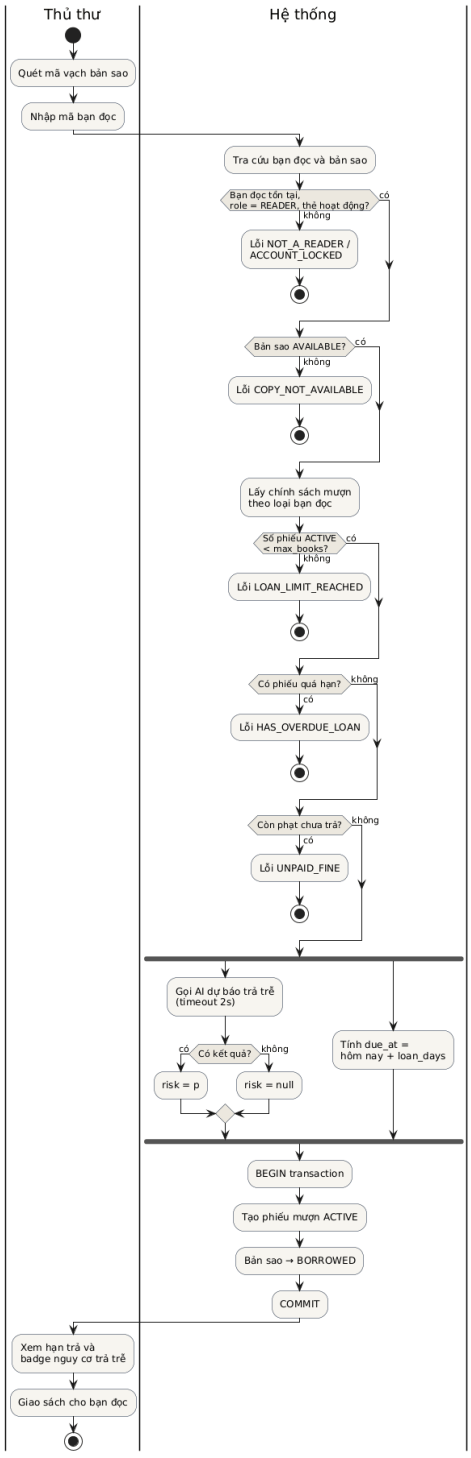
Hình 5. Sequence: gợi ý sách cá nhân hoá (US18)



Hình 6. Sequence: đặt trước sách và giữ bản sao khi có người trả (US21, US24)



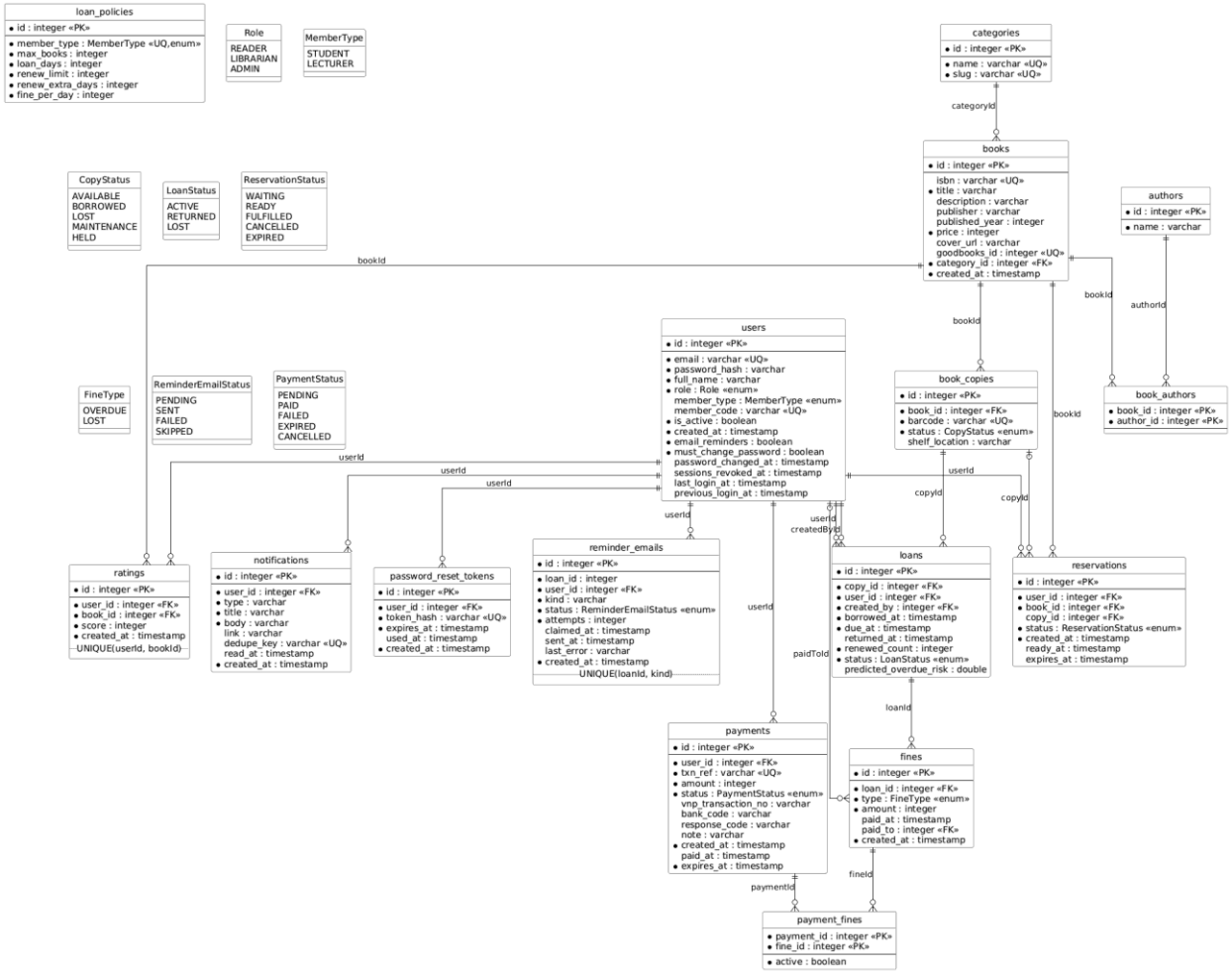
Hình 7. Sequence: nộp phạt trực tuyến qua VNPay (US30)



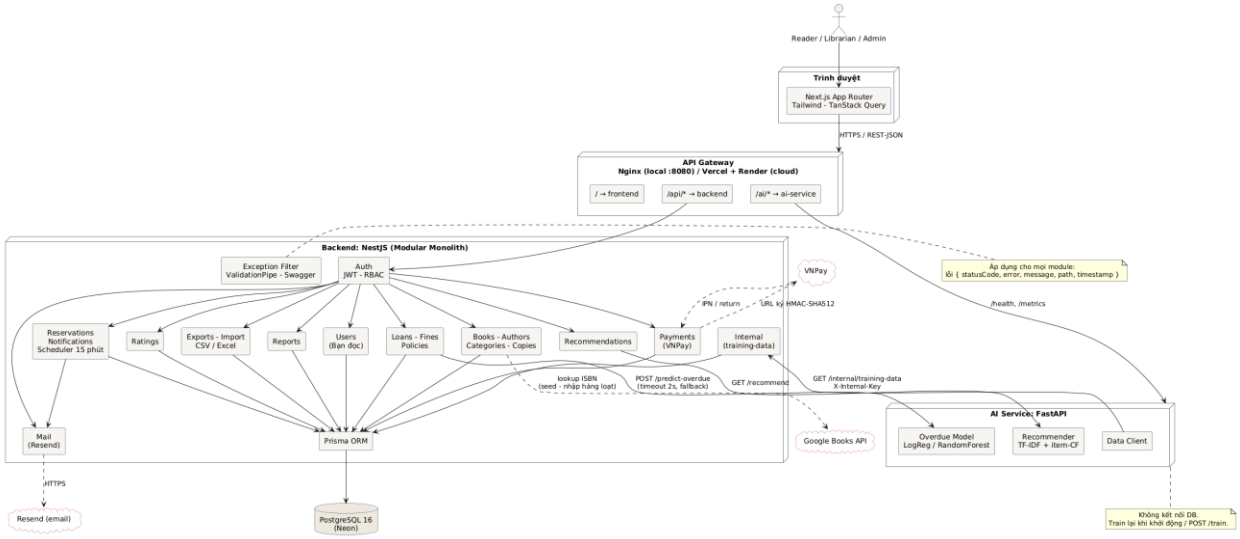
Hình 8. Activity Diagram: quy trình cho mượn sách tại quầy

d) Thiết kế Cơ sở dữ liệu (Database ERD): ERD sinh tự động từ Prisma schema, 16 bảng chuẩn 3NF. Bảng chính: `users` (tài khoản 3 vai trò), `loan_policies` (chính sách mượn theo loại bạn đọc), `books`, `authors`, `categories`, `book_authors` (quan hệ n-

n), `book_copies` (bản sao có mã vạch, trạng thái, gồm `HELD` khi giữ cho người đặt trước), `loans` (phiếu mượn, hạn trả, nguy cơ trả trễ), `fines` (phạt `OVERDUE` và `LOST`), `ratings`, `reservations` (đặt trước), `notifications`, `reminder_emails`, `password_reset_tokens`, `payments` và `payment_fines` (thanh toán VNPay). DDL đầy đủ: `docs/erd/ddl.sql`; sơ đồ kiến trúc và đặc tả API (Method, URL, Request Body, Response Code, mã lỗi) trong `docs/design/SDD.md`.



Hình 9. ERD của cơ sở dữ liệu (16 bảng)



Hình 10. Sơ đồ kiến trúc: gateway, các dịch vụ, cơ sở dữ liệu, tích hợp ngoài

2.3. LAB 03: Phát triển Chức năng Cốt lõi & Tích hợp AI/ML

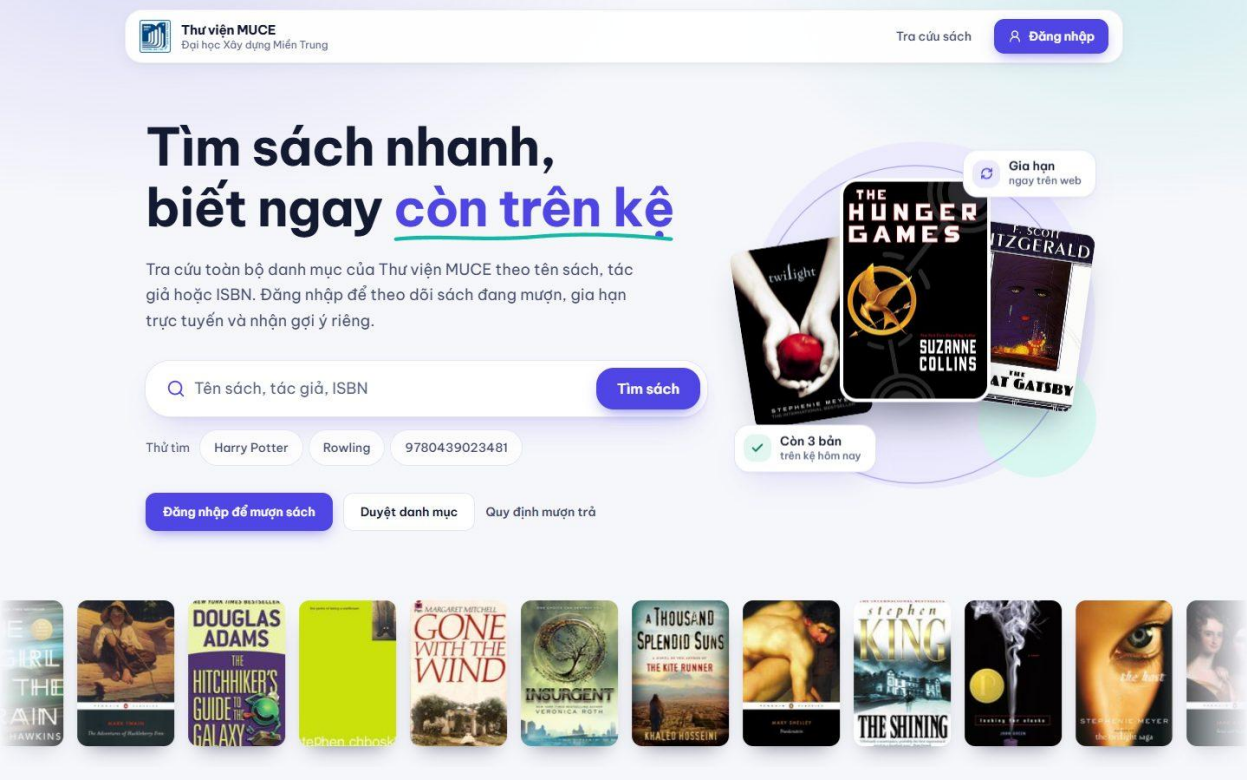
a) Kết quả phát triển Backend API: Mô tả công nghệ sử dụng và các Endpoint chính đã hoàn thiện.

- **Công nghệ:** NodeJS NestJS 10, Prisma 6, PostgreSQL 16; xác thực JWT (Passport), phân quyền RBAC 3 vai trò; Swagger tự sinh tại `/api/docs`.
- **Đã hoàn thiện 78 RESTful API** (đếm trong OpenAPI của hệ thống đang chạy) thuộc 18 nhóm: auth, users, policies, books, authors, categories, copies, loans, fines, ratings, reports, reservations, notifications, payments, recommendations, exports, internal, health.
- **Kiến trúc** Controller, Service, Repository (Prisma); toàn bộ quy tắc nghiệp vụ (BR-01 đến BR-26) nằm ở tầng Service. Mọi lỗi trả về một định dạng thống nhất `{ statusCode, error, message, path, timestamp }` kèm mã nghiệp vụ rõ ràng: `LOAN_LIMIT_REACHED`, `HAS_OVERDUE_LOAN`, `UNPAID_FINE`, `COPY_NOT_AVAILABLE`, `RESERVATION_LIMIT`, `FINE_PAYMENT_PENDING`.
- **Chống đua dữ liệu:** mượn, trả, gia hạn, báo mất, đặt trước, thu phạt và thanh toán dùng khoá hàng bạn đọc (`FOR UPDATE`), cập nhật có điều kiện và `SKIP LOCKED` khi chọn người đứng đầu hàng đợi; bộ kiểm thử đua bắn yêu cầu song song trên PostgreSQL thật (ví dụ năm bạn đọc mượn cùng một bản sao thì đúng một phiếu thành công).

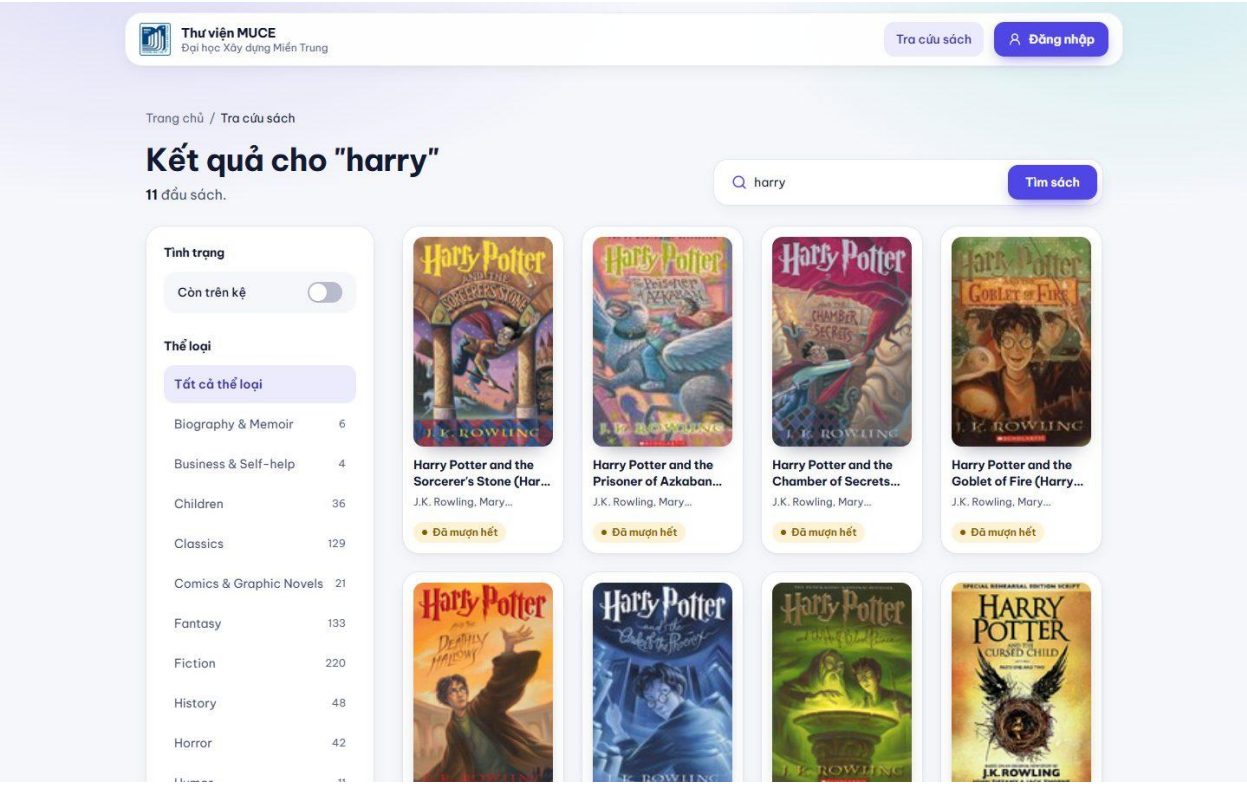
- **Tích hợp ngoài:** Google Books (nhập sách theo ISBN), Resend (email quên mật khẩu, nhắc hạn), VNPay (thanh toán phạt, ký và kiểm tra HMAC-SHA512, xử lý IPN và return idempotent, chặn thu trùng).
- **Bảo mật:** mật khẩu băm bcrypt, JWT hạn 24 giờ và đọc lại tài khoản ở mỗi yêu cầu (khoá thẻ có hiệu lực ngay), giới hạn tần suất theo IP, kiểm tra đầu vào, chống formula injection khi xuất CSV.

b) Kết quả giao diện Frontend UI/UX: ảnh chụp từ hệ thống đang chạy tại https://muce-library.vercel.app.

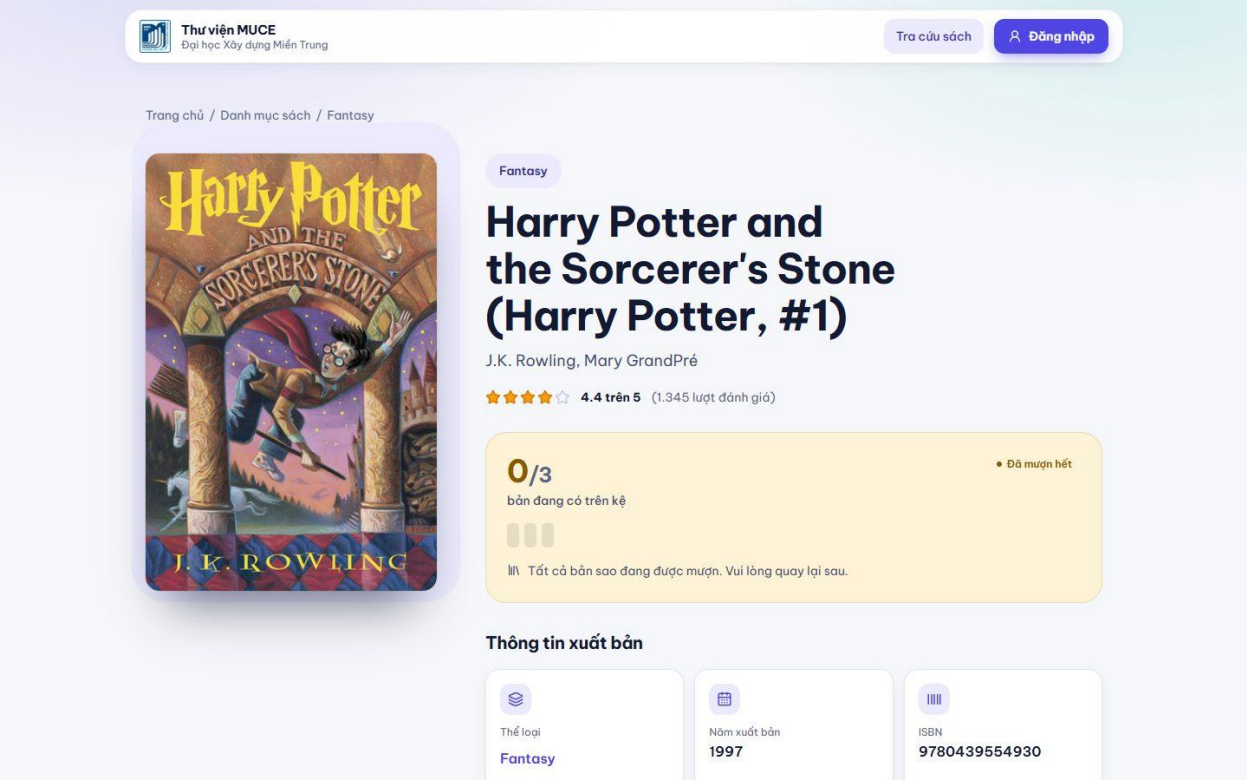
- **Công nghệ:** Next.js 15 App Router, TypeScript, TailwindCSS 4, TanStack Query, Recharts (nạp theo yêu cầu), phông Be Vietnam Pro; 29 trang cho ba nhóm người dùng (bạn đọc, thủ thư, quản trị).
- **Thiết kế:** giao diện sáng, bo tròn lớn, hoạt ảnh chỉ dùng `transform` và `opacity` để mượt, tôn trọng thiết lập giảm chuyển động của hệ điều hành.
- **SEO và khả năng tiếp cận:** trang công khai dựng phía máy chủ, tiêu đề và mô tả riêng từng trang, dữ liệu có cấu trúc schema.org, sitemap và robots tự sinh; mỗi trang một thẻ h1, nhãn cho mọi ô nhập, viền focus rõ ràng.
- **Responsive:** kiểm ở bề rộng 390, 768 và 1440 px, không cuộn ngang; bảng dữ liệu chuyển thành thẻ dọc trên điện thoại; thanh điều hướng thu gọn thành nút menu. Quyền thật vẫn do backend quyết định.



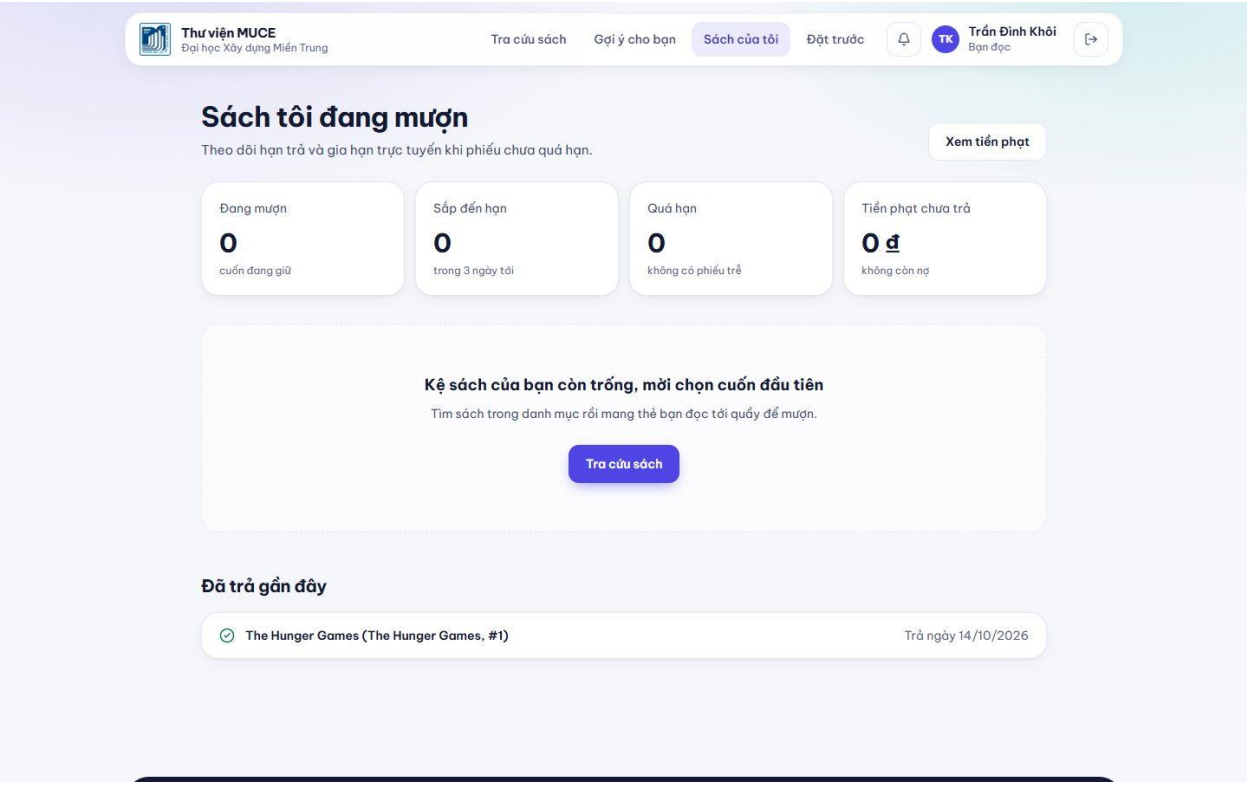
Hình 11. Trang chủ: ô tìm kiếm, từ khoá gợi ý và sách còn trên kệ



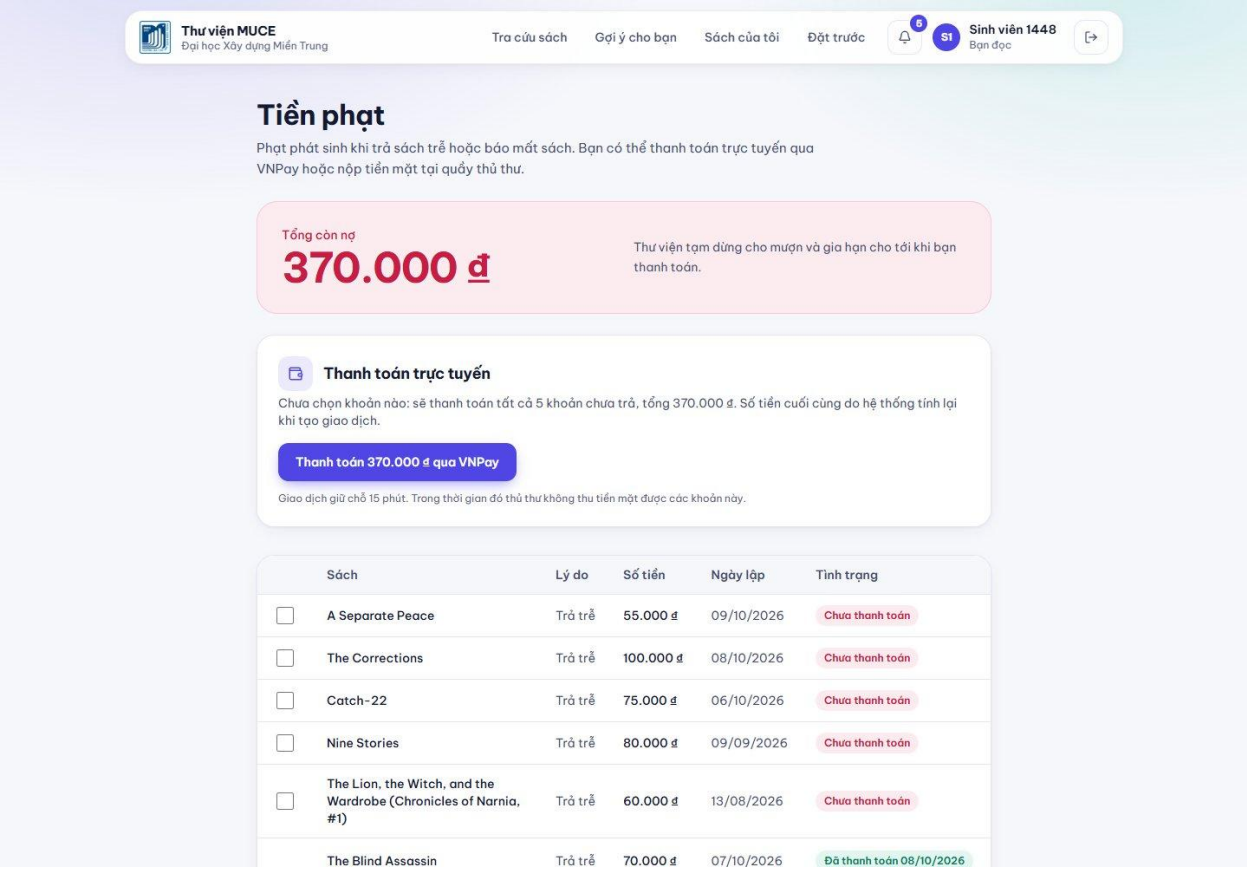
Hình 12. Tra cứu sách với bộ lọc thể loại và tình trạng



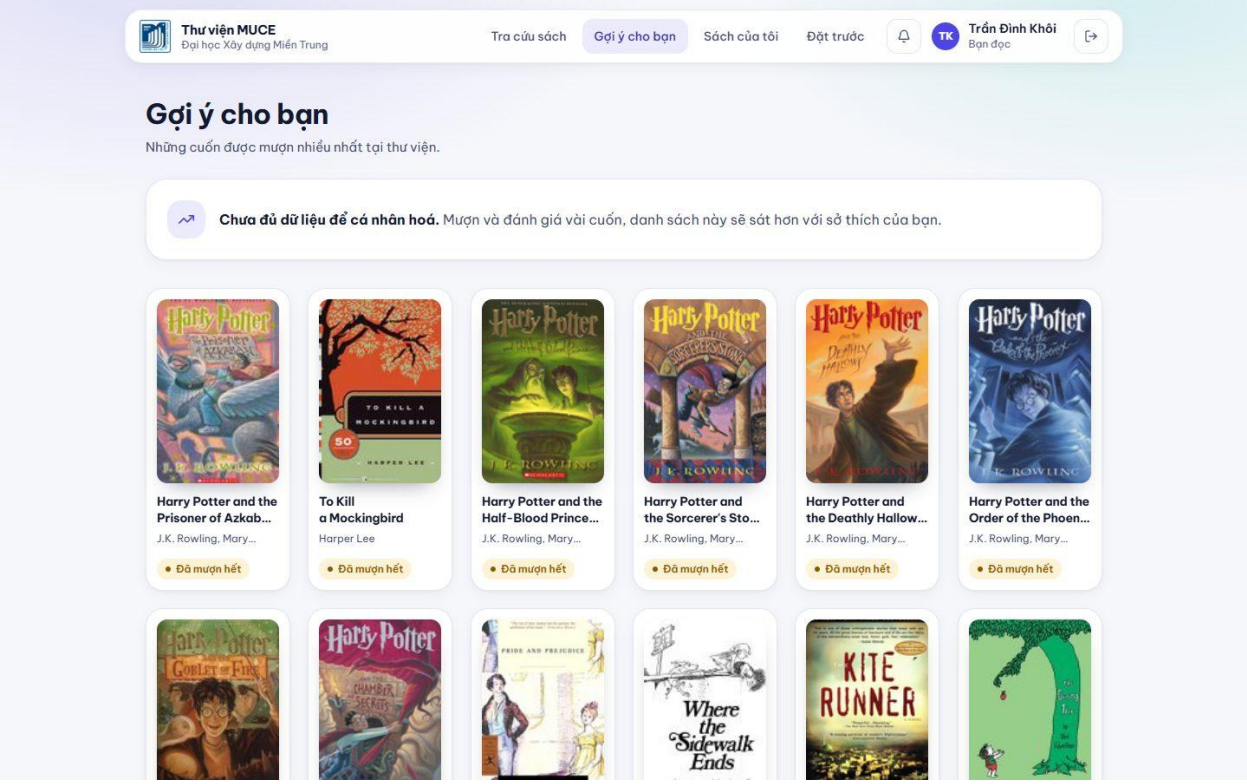
Hình 13. Chi tiết sách kèm tình trạng bản sao và điểm đánh giá



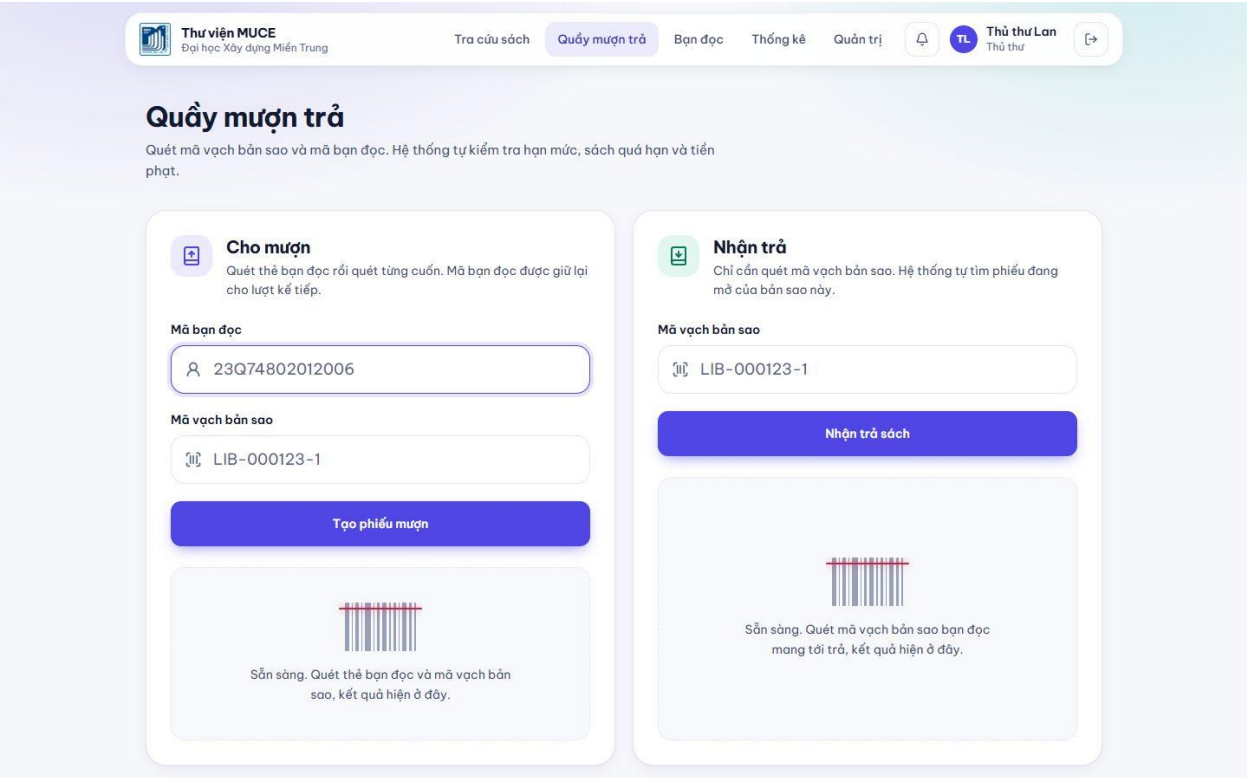
Hình 14. Bạn đọc xem sách đang mượn, hạn trả và lịch sử



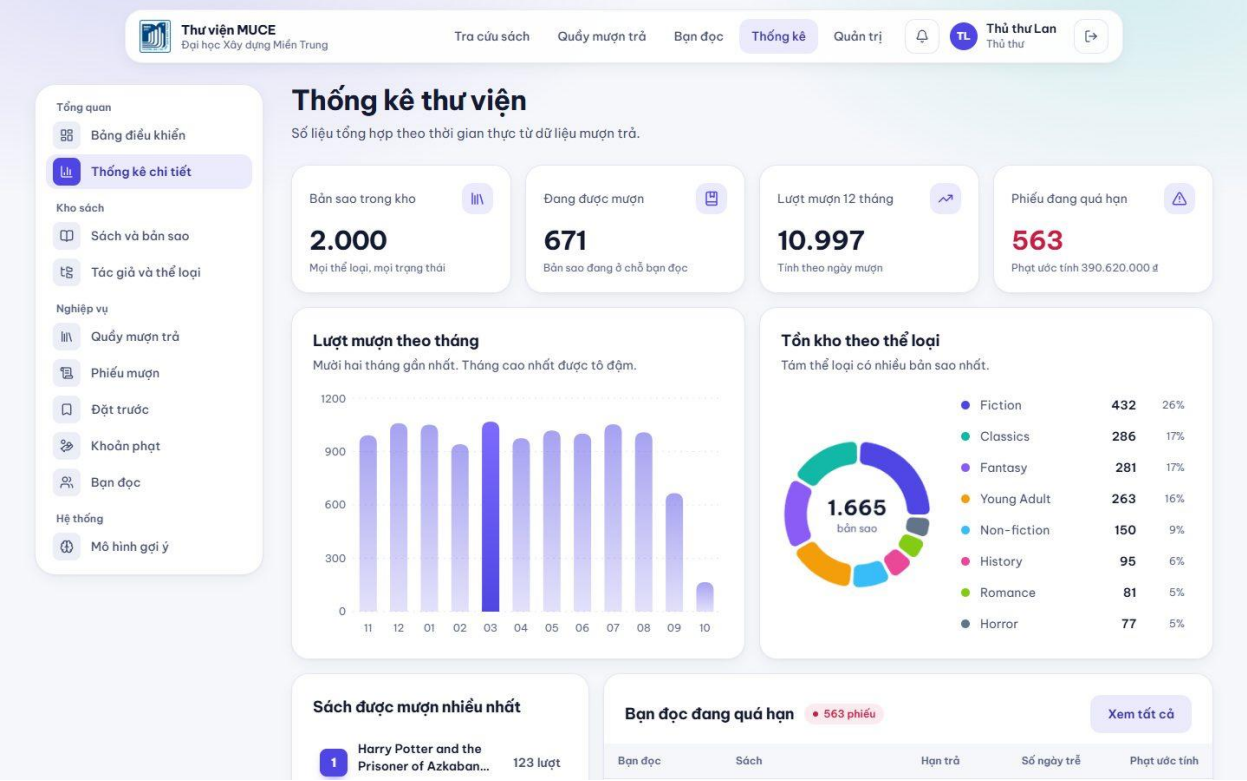
Hình 15. Trang tiền phạt và thanh toán trực tuyến qua VNPay (tổng nợ, các khoản phạt, nút thanh toán)



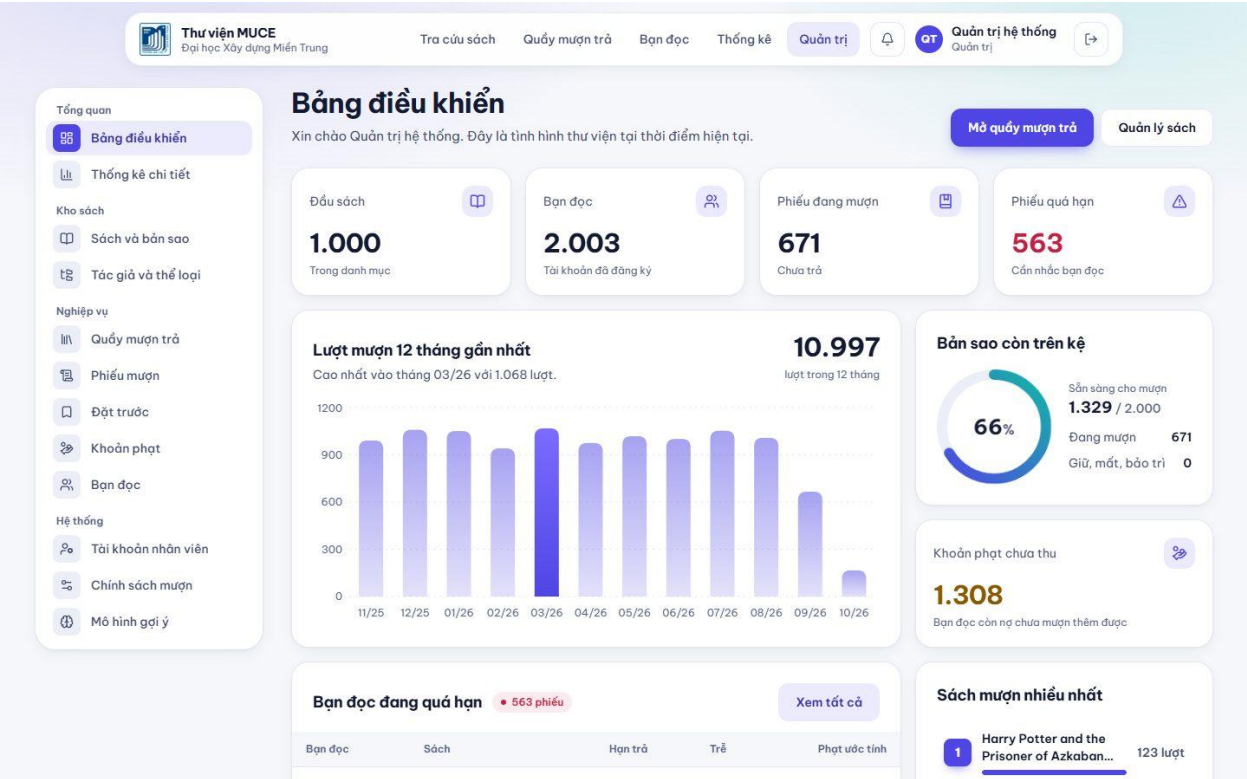
Hình 16. Trang gợi ý sách cho bạn đọc



Hình 17. Quầy mượn trả của thủ thư (quét mã vạch bản sao và mã bạn đọc)



Hình 18. Thống kê thư viện với biểu đồ lượt mượn và tồn kho



Hình 19. Bảng điều khiển quản trị



Hình 20. Giao diện trên điện thoại, bề rộng 390 px

c) Module AI/ML Tích hợp: Mô tả thuật toán và kết quả thực nghiệm (số đo lấy từ `GET /api/recommendations/metrics` của hệ thống đang chạy, huấn luyện ngày 19/10/2026).

- **Mô hình 1, gợi ý sách (lai):** TF-IDF trên tiêu đề, tác giả, thể loại, mô tả (cosine similarity) trộn 50/50 với Collaborative Filtering item-based trên ma trận bạn đọc và sách. Đo trên tập giữ lại 20%: Precision@5 = 0,309 và Recall@5 = 0,110 (1.000 sách, 2.000 bạn đọc, 122.354 tương tác, 300 bạn đọc được đánh giá), vượt ngưỡng nghiệm thu 0,15.
- **Mô hình 2, dự báo nguy cơ trả trễ:** so sánh Logistic Regression với Random Forest trên 10.859 phiếu đã trả (tỉ lệ trễ 60,1%), tách 20% làm tập kiểm tra. Logistic Regression: Accuracy 0,709, Precision 0,789, Recall 0,703, F1 0,743, ROC-AUC 0,788. Random Forest: Accuracy 0,719, Precision 0,786, Recall 0,730, F1 0,757,

ROC-AUC 0,784. Hệ thống tự chọn Logistic Regression vì ROC-AUC cao hơn; đạt ngưỡng 0,70.

- **Đặc trưng:** số lần mượn trước đó, số lần và tỉ lệ trả trễ trước đó, số sách đang giữ, loại bạn đọc, thể loại sách, số ngày mượn, tháng mượn và cờ mùa thi; dựng theo trình tự thời gian nên mỗi phiếu chỉ dùng thông tin có trước ngày mượn.
- **Lưu ý trung thực:** lịch sử mượn trả dùng để huấn luyện là dữ liệu mô phỏng sinh từ bộ goodbooks-10k, nên số đo phản ánh quy luật mô phỏng, chưa phải hành vi bạn đọc thật.
- **Chịu lỗi:** khi AI Service không phản hồi, việc cho mượn vẫn thành công với nguy cơ trả trễ rỗng (`null`), trang gợi ý chuyển sang danh sách sách mượn nhiều, các điểm cuối quản trị AI trả 503 `AI_UNAVAILABLE` thay vì lỗi 500.

Snippet Tích hợp AI Model (Python/REST API)

```
# Snippet 1. ai-service/app/overdue_model.py: chọn mô hình tốt hơn theo ROC-AUC
logistic = Pipeline([("scaler", StandardScaler()),
                     ("clf", LogisticRegression(max_iter=1000,
class_weight="balanced"))
                    ]).fit(x_train, y_train)
forest = RandomForestClassifier(n_estimators=200, max_depth=12, min_samples_leaf=5,
                                class_weight="balanced",
random_state=42).fit(x_train, y_train)
logistic_scores = score_model(logistic, x_test, y_test)
forest_scores = score_model(forest, x_test, y_test)
chosen_is_forest = forest_scores.roc_auc >= logistic_scores.roc_auc
self.model = forest if chosen_is_forest else logistic

// Snippet 2. backend/src/modules/ai/ai.client.ts: AI chậm không được chặn quầy
mượn (FR-LOAN-02)
const res = await this.fetchImpl(`${this.baseUrl}/predict-overdue`, {
  method: 'POST',
  signal: AbortSignal.timeout(this.predictTimeoutMs),   // 2 giây
});
// lỗi hoặc quá hạn chờ: trả null, phiếu mượn vẫn được tạo bình thường
```

Bảng 3. Snippet tích hợp mô hình AI

2.4. LAB 04: Containerization, CI/CD Pipeline & Triển khai Cloud

a) Đóng gói Container Docker: Toàn bộ hệ thống khởi chạy bằng một lệnh `docker compose up --build`, gồm 5 container: postgres (cơ sở dữ liệu), backend (NestJS), ai-service (FastAPI), frontend (Next.js) và gateway (Nginx, cổng 8080). Backend tự áp migration khi khởi động; frontend, backend và AI Service đều có Dockerfile riêng. Job

compose-smoke trong CI chạy đúng lệnh này trên máy ảo sạch rồi gọi thử ba địa chỉ `/health` qua gateway. Nội dung `docker-compose.yml`:

Cấu hình `docker-compose.yml`

```
name: uls

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: uls
      POSTGRES_PASSWORD: uls
      POSTGRES_DB: uls
    ports:
      - "${POSTGRES_PORT:-5434}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U uls -d uls"]
      interval: 5s
      timeout: 3s
      retries: 10

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://uls:uls@postgres:5432/uls?schema=public
      PORT: 3000
      JWT_SECRET: ${JWT_SECRET:-dev-secret}
      CORS_ORIGINS: http://localhost:${GATEWAY_PORT:-8080}
      AI_SERVICE_URL: http://ai-service:8000
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-dev-internal}
      GOOGLE_BOOKS_API_KEY: ${GOOGLE_BOOKS_API_KEY:-}
      RESEND_API_KEY: ${RESEND_API_KEY:-}
      MAIL_FROM: ${MAIL_FROM:-}
      APP_PUBLIC_URL: ${APP_PUBLIC_URL:-http://localhost:${GATEWAY_PORT:-8080}}
      VNPAY_TMN_CODE: ${VNPAY_TMN_CODE:-}
      VNPAY_HASH_SECRET: ${VNPAY_HASH_SECRET:-}
      VNPAY_URL: ${VNPAY_URL:-https://sandbox.vnpayment.vn/paymentv2/vpcpay.html}
    depends_on:
      postgres:
        condition: service_healthy

  ai-service:
    build: ./ai-service
    environment:
```

Cấu hình `docker-compose.yml`

```
      BACKEND_URL: http://backend:3000/api
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-dev-internal}
      TRAIN_ON_STARTUP: ${TRAIN_ON_STARTUP:-1}
    depends_on:
      - backend

  frontend:
    build: ./frontend
    environment:
      API_URL: http://backend:3000/api
      NEXT_PUBLIC_SITE_URL: ${PUBLIC_SITE_URL:-http://localhost:8080}
    depends_on:
      - backend

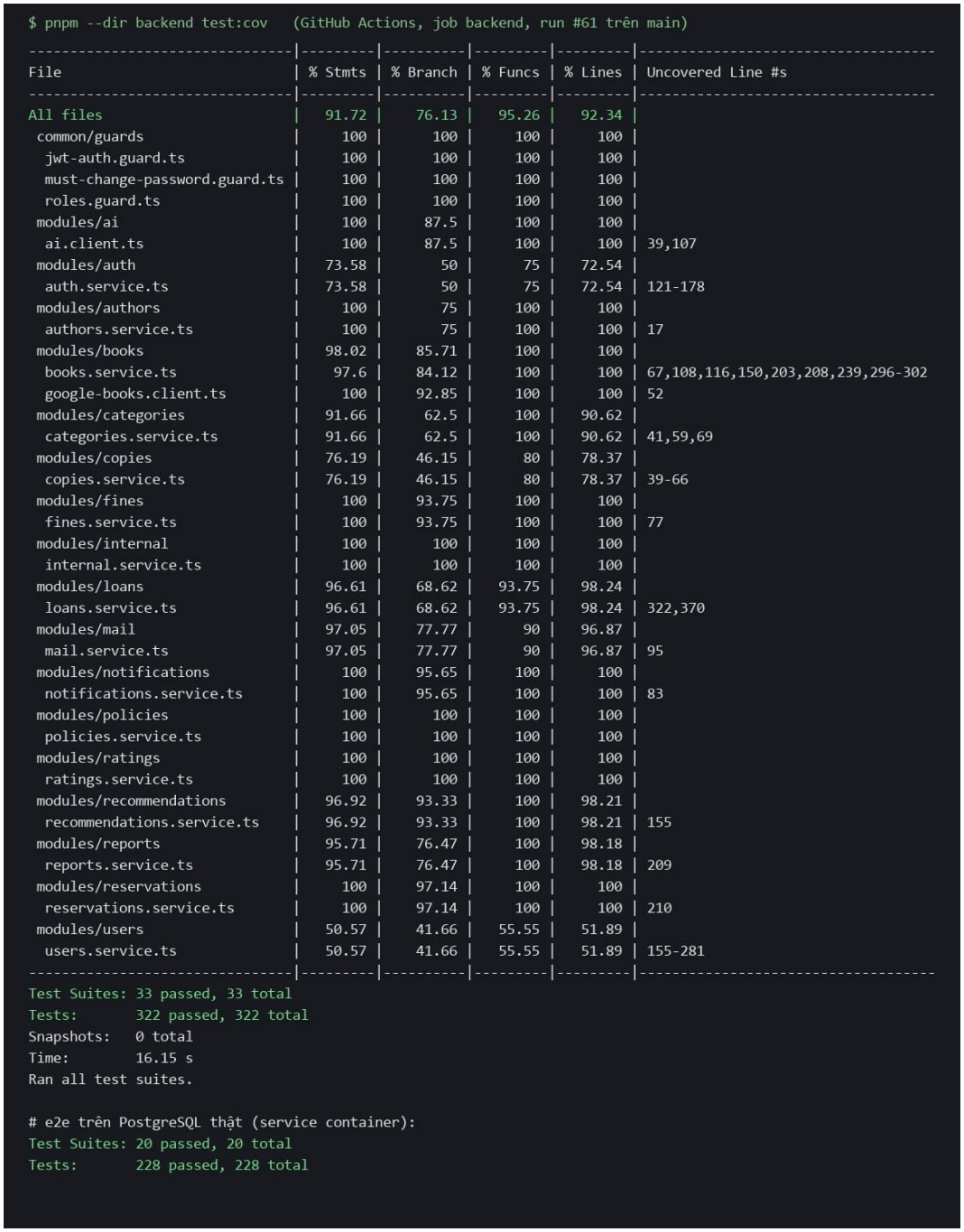
  gateway:
    image: nginx:1.27-alpine
    volumes:
      - ./gateway/nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "${GATEWAY_PORT:-8080}:80"
    depends_on:
      - frontend
      - backend
      - ai-service

volumes:
  pgdata:
```

Bảng 4. Cấu hình `docker-compose.yml`

b) Kết quả Kiểm thử Unit Test: Kết quả thực thi Unit Test và độ bao phủ mã nguồn (Code Coverage):

- **Tổng số Test Cases: 576, tất cả đều đạt (100%)** ở lần chạy GitHub Actions run #61 trên main (19/10/2026): 322 unit test backend (Jest, 33 suite), 228 e2e test backend chạy trên PostgreSQL thật (20 suite, gồm các ca đua song song) và 26 test AI Service (pytest).
- **Độ bao phủ mã nguồn (Code Coverage):** backend (unit test, đo trên lớp nghiệp vụ) đạt 91,72% câu lệnh, 76,13% nhánh, 95,26% hàm, 92,34% dòng; AI Service đạt 93,59%. Ngưỡng được ép trong cấu hình: backend 80% dòng và 75% nhánh (năm service nặng SQL như thanh toán, xuất dữ liệu, nhập sách, email nhắc hạn, đặt lại mật khẩu được kiểm bằng e2e riêng nên không tính vào thước đo unit), AI Service 80%; dưới ngưỡng thì pipeline đỏ.



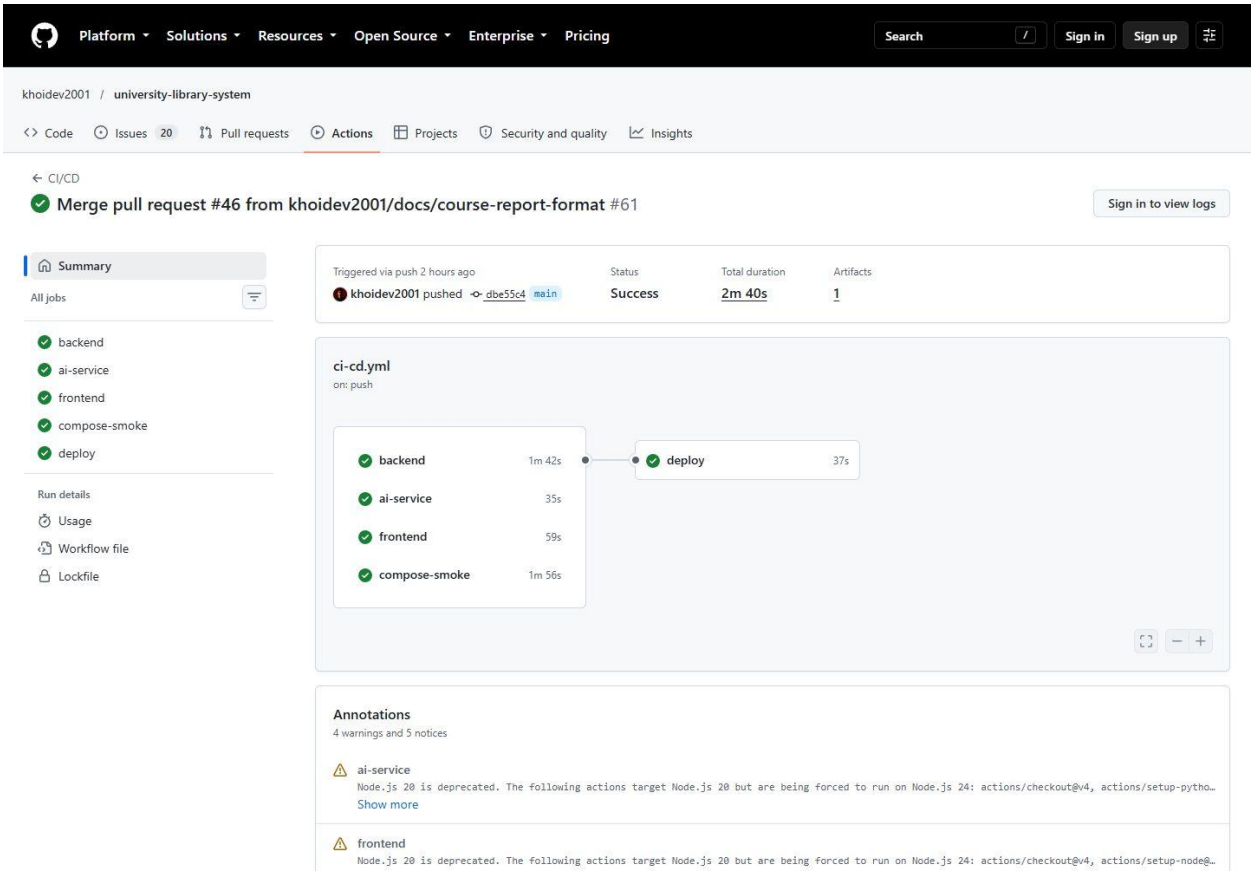
```
$ pnpm --dir backend test:cov   (GitHub Actions, job backend, run #61 trên main)
-------------------------------|---------|----------|---------|---------|-----------------------------------
File                           | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------------------|---------|----------|---------|---------|-----------------------------------
All files                      |   91.72 |    76.13 |   95.26 |   92.34 |
 common/guards                 |     100 |      100 |     100 |     100 |
  jwt-auth.guard.ts            |     100 |      100 |     100 |     100 |
  must-change-password.guard.ts |     100 |      100 |     100 |     100 |
  roles.guard.ts               |     100 |      100 |     100 |     100 |
 modules/ai                    |     100 |     87.5 |     100 |     100 |
  ai.client.ts                 |     100 |     87.5 |     100 |     100 | 39,107
 modules/auth                  |   73.58 |       50 |      75 |   72.54 |
  auth.service.ts              |   73.58 |       50 |      75 |   72.54 | 121-178
 modules/authors               |     100 |       75 |     100 |     100 |
  authors.service.ts           |     100 |       75 |     100 |     100 | 17
 modules/books                 |   98.02 |    85.71 |     100 |     100 |
  books.service.ts             |    97.6 |    84.12 |     100 |     100 | 67,108,116,150,203,208,239,296-302
  google-books.client.ts       |     100 |    92.85 |     100 |     100 | 52
 modules/categories            |   91.66 |     62.5 |     100 |   90.62 |
  categories.service.ts        |   91.66 |     62.5 |     100 |   90.62 | 41,59,69
 modules/copies                |   76.19 |    46.15 |      80 |   78.37 |
  copies.service.ts            |   76.19 |    46.15 |      80 |   78.37 | 39-66
 modules/fines                 |     100 |    93.75 |     100 |     100 |
  fines.service.ts             |     100 |    93.75 |     100 |     100 | 77
 modules/internal              |     100 |      100 |     100 |     100 |
  internal.service.ts          |     100 |      100 |     100 |     100 |
 modules/loans                 |   96.61 |    68.62 |   93.75 |   98.24 |
  loans.service.ts             |   96.61 |    68.62 |   93.75 |   98.24 | 322,370
 modules/mail                  |   97.05 |    77.77 |      90 |   96.87 |
  mail.service.ts              |   97.05 |    77.77 |      90 |   96.87 | 95
 modules/notifications         |     100 |    95.65 |     100 |     100 |
  notifications.service.ts     |     100 |    95.65 |     100 |     100 | 83
 modules/policies              |     100 |      100 |     100 |     100 |
  policies.service.ts          |     100 |      100 |     100 |     100 |
 modules/ratings               |     100 |      100 |     100 |     100 |
  ratings.service.ts           |     100 |      100 |     100 |     100 |
 modules/recommendations       |   96.92 |    93.33 |     100 |   98.21 |
  recommendations.service.ts   |   96.92 |    93.33 |     100 |   98.21 | 155
 modules/reports               |   95.71 |    76.47 |     100 |   98.18 |
  reports.service.ts           |   95.71 |    76.47 |     100 |   98.18 | 209
 modules/reservations          |     100 |    97.14 |     100 |     100 |
  reservations.service.ts      |     100 |    97.14 |     100 |     100 | 210
 modules/users                 |   50.57 |    41.66 |   55.55 |   51.89 |
  users.service.ts             |   50.57 |    41.66 |   55.55 |   51.89 | 155-281
-------------------------------|---------|----------|---------|---------|-----------------------------------
Test Suites: 33 passed, 33 total
Tests:       322 passed, 322 total
Snapshots:   0 total
Time:        16.15 s
Ran all test suites.

# e2e trên PostgreSQL thật (service container):
Test Suites: 20 passed, 20 total
Tests:       228 passed, 228 total
```

Hình 21. Báo cáo coverage của backend trích từ log job backend trên GitHub Actions (run #61)

c) Cấu hình CI/CD Pipeline: Workflow `.github/workflows/ci-cd.yml` tự chạy mỗi khi push hoặc mở Pull Request, gồm 4 job song song và 1 job triển khai. **backend:** lint, build, unit test kèm ngưỡng coverage, e2e test trên PostgreSQL thật (service container). **ai-service:** pytest kèm ngưỡng coverage. **frontend:** lint và build. **compose-smoke:** dựng cả 5 container trên máy ảo sạch và gọi thử qua gateway. **deploy:** chỉ chạy

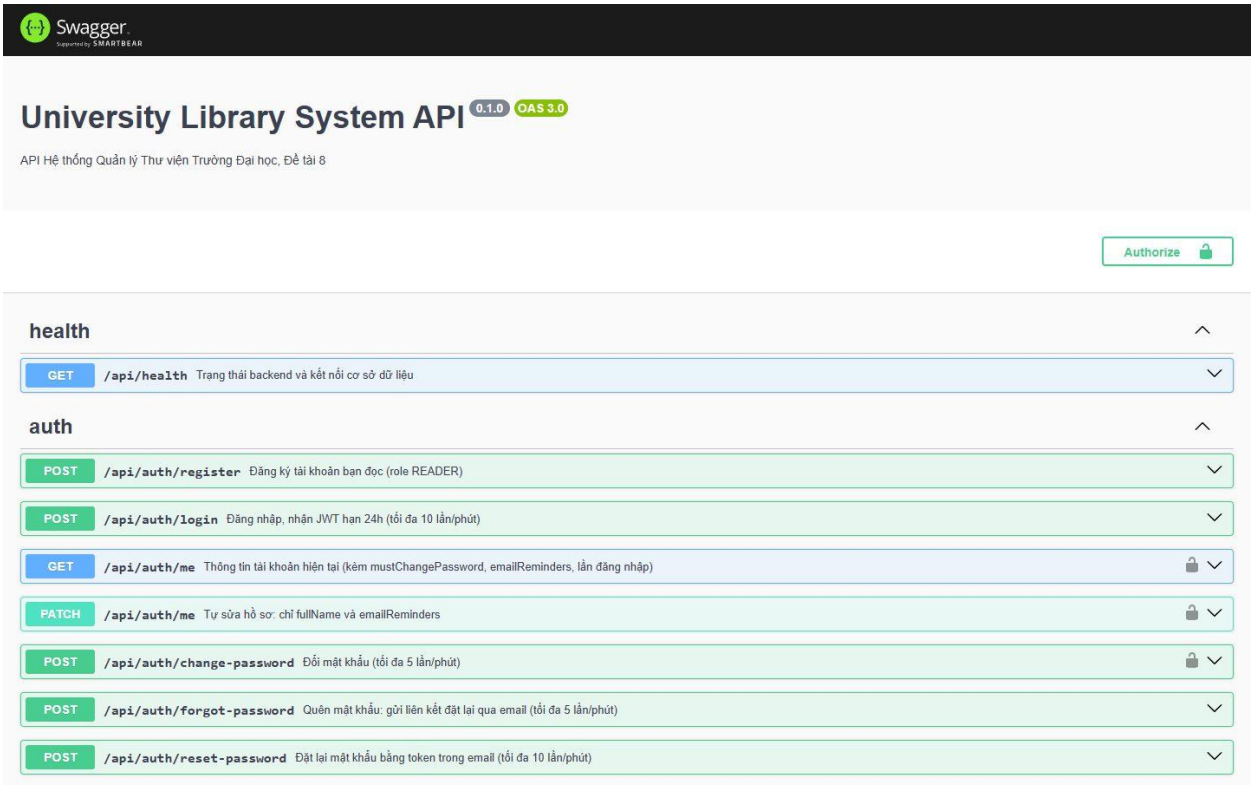
khi gộp vào main và mọi job trước đó xanh, gọi Deploy Hook của Render rồi kiểm tra các địa chỉ trực tuyến.



Hình 22. Kết quả pipeline GitHub Actions (run #61 trên main, 4 job đạt và job deploy)

d) Triển khai Cloud: Kiến trúc triển khai gồm Vercel (frontend), Render (backend và AI Service, Docker) và Neon (PostgreSQL), đều dùng gói miễn phí. Cấu hình nằm trong repo: `render.yaml`, `frontend/next.config.ts` (chuyển tiếp `/api` và `/ai` sang Render để trình duyệt chỉ nói chuyện với một địa chỉ) và hướng dẫn từng bước `docs/deploy/DEPLOY.md`. Hệ thống đã chạy thật trên cloud. Gói miễn phí tự ngủ khi không có truy cập nên lần mở đầu có thể mất khoảng một phút. Địa chỉ truy cập ứng dụng trực tuyến:

- URL Web Application: https://muce-library.vercel.app
- URL API Swagger Docs: https://muce-library.vercel.app`/api/docs` (trực tiếp trên Render: https://uls-backend-bo6w.onrender.com`/api/docs`)



Hình 23. Swagger UI của hệ thống đang chạy trên cloud

PHẦN III: KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

3.1. Các kết quả chính đã đạt được

- Hoàn thiện cả 04 bài thực hành LAB theo tiến trình Scrum: Product Backlog 37 User Story (155 điểm), gồm 20 story MVP trong 4 sprint và 17 story mở rộng ở sprint 5 (đặt trước, thông báo và email nhắc hạn, tài khoản và mật khẩu, thanh toán VNPay, xuất báo cáo và in thẻ, nhập sách từ tệp); tất cả chạy được trên hệ thống thật.
- Hệ thống gồm 78 RESTful API, 29 trang giao diện responsive chuẩn SEO, hai mô hình AI đạt ngưỡng nghiệm thu (Precision@5 = 0,309 so với ngưỡng 0,15; ROC-AUC = 0,788 so với ngưỡng 0,70), 576 test tự động đạt 100% trên CI.
- Áp dụng quy trình DevOps tự động hoá: một lệnh `docker compose up` dựng toàn bộ hệ thống, CI chạy đủ kiểm thử trên mỗi Pull Request và chỉ triển khai khi xanh, tài liệu thiết kế (ERD, đặc tả API) sinh từ mã nguồn nên luôn khớp với hệ thống.

3.2. Hạn chế và Hướng phát triển tiếp theo

- **Hạn chế:** (1) Lịch sử mượn trả dùng để huấn luyện AI là dữ liệu mô phỏng nên độ chính xác phản ánh quy luật mô phỏng. (2) Sách mẫu lấy từ goodbooks-10k nên tên sách tiếng Anh và chưa có mô tả; nhập sách theo ISBN từ Google Books đã có mã nguồn và unit test nhưng chưa thử với dữ liệu thật do giới hạn tần suất. (3) Frontend chưa có test tự động, mới kiểm bằng lint, build và kiểm tay. (4) Thanh toán VNPay mới ở môi trường sandbox; email chỉ gửi tới được các địa chỉ đã xác minh trên Resend vì chưa có tên miền riêng. (5) JWT hạn một ngày, chưa có refresh token. (6) Hạ tầng miễn phí không có SLA: Render ngủ sau 15 phút không dùng nên lần gọi đầu mất khoảng một phút.
- **Hướng phát triển:** thay dữ liệu mô phỏng bằng dữ liệu mượn trả thật của thư viện rồi huấn luyện lại, so sánh thêm gradient boosting; xác thực tên miền gửi email và chuyển VNPay sang môi trường thật; bổ sung test giao diện bằng Playwright và kiểm thử tải để đo chỉ tiêu 2 giây ở p95; thêm nhật ký kiểm toán, cơ chế đồng ý và quyền của chủ thể dữ liệu cá nhân; lưu mô hình ra bộ nhớ ngoài để AI Service khởi động nhanh sau khi thức dậy.